

Operating AI Agents at Scale — Cost, Latency, Scale & Lifecycle

A reference for running a production agent platform under load and under change. Two control loops, one for demand and resources and one for behavioural releases, tested in a deterministic simulator with ten scenarios, each naive against controlled, recorded once and replayed byte for byte.

O

O1 + O2 · Production AI Engineering

58 min read · Published 2026-10-08 · Technical deep dive

— PRODUCTION AI ENGINEERING · 01 + 02 · SCALE & OPERATIONS

Operating AI Agents at Scale Cost, Latency, Scale & Lifecycle

your agent survived Black Friday.
then someone changed the prompt.

At production scale, cost and latency become architectural constraints,
and every agent change becomes an operational event.



RUNTIME CONTROL

Black Friday: a surge on one shared platform

max work in system	5,791	→	128
goodput %	9.3	→	45.5

open admission → bounded admission



CHANGE CONTROL

Cyber Monday: one tool description, no code

HTTP-style errors	+0.0 pp
cost per success	-0.1%
tool calls per workflow	+88.9%

canary: ROLLBACK

same platform · same workload · only the controls differ

run 2026-10-08-recorded · simulation units

The reference companion to the Medium edition. Every number is substituted from the recorded run and is a simulation unit; every external claim links to a numbered source fetched and quoted in `research/sources.md`.

About this edition

The Medium edition makes one argument: **at production scale, cost and latency become architectural constraints, and every agent change becomes an operational event**. Operating an agent platform is not keeping agents available. It is controlling demand, resources, behaviour and change. This edition is the reference behind that argument, for platform engineers, SREs and architects who run agents for many users and keep changing them. It covers:

- **workload amplification**: why one request is many units of work, and what that does to capacity planning;
- the **workflow resource envelope**, and how tenant budgets, workflow budgets, provider quotas and downstream quotas differ;
- **admission, fair scheduling, bounded concurrency, deadlines and backpressure** as orchestration's job;
- **routing inside an eligibility contract**, provider quotas and fallback that cannot cross a policy;
- the **operational cost of context** and cache scope;
- **tool capacity** as its own limit, and why controls compose;
- **cost attribution**, cost per successful outcome, and latency decomposition;
- the **behavioural release**: what an agent release contains, why “LLMOps = prompt versioning” is too small, and a manifest whose id changes when behaviour can;
- **the change control loop**: offline evals, binary invariant gates, staging, safe shadowing, a behavioural canary, promotion and a rollback that is not `kubectl rollout undo`;
- a recorded POC: 10 scenarios, each a naive arm against a controlled arm on the same workload, 65 preregistered checks, a negative control, and a byte-identical replay.

HOW TO READ THE FIGURES

- requests, demand, operations ■ orchestration, the runtime loop
- the agent and its models: probabilistic ■ context and retrieval ■ tools and downstream APIs
- deterministic controls: admission, bounds, gates ■ control plane, releases, the change loop
- evidence ■ enterprise systems (simulated) ■ the naive arm, a breach, a rollback
- the controlled arm, held, promoted

ARCHITECTURE conceptual design **MEASURED** a recorded POC result, in simulation units

RECORDED one recorded decision

Statements are marked by what they rest on:

- **Sourced.** An established concept with a numbered reference, such as SRE's observation that “Queued requests consume memory and increase latency” ^[2]. Every source is quoted in `research/sources.md` with what it does *not* support.
- **Our synthesis.** A position this series takes. The *workflow resource envelope*, the *behavioural release manifest*, the *two control loops* and *cost per successful outcome* are in this category: no source defines them. The nearest sourced terms are Anthropic's “cost per task” ^[41] and OpenTelemetry's per-invocation call counts ^[23].
- **Implemented / measured / recorded.** Behaviour of this article's POC, a deterministic simulator. Numbers are substituted at build time from `ops_poc/runs/2026-10-08-recorded/facts.json` and are **simulation units** (simulated milliseconds and cost units, cu). They show what a mechanism does under a declared workload. They are not benchmarks of any provider, model or platform.

1 • Executive summary

An agent that works for ten users a day meets two new problems in production. **Load:** requests become workflows, workflows become model calls, retrieval queries and tool calls; queues grow, providers throttle, downstream systems saturate and tenants interfere. **Change:** the system keeps changing, often without a code deploy: a prompt, a model, a routing table, a tool description, a retrieval setting, a knowledge index, a policy. Both converge on operations.

The architecture this series built (P1's platform, with F2's layers and T4's control plane) already has a place for every control this note needs. Nothing new is added; the note says which existing layer owns which control and tests whether the controls hold.

Production AI operations require two control loops: one that continuously governs runtime demand and resources, and another that governs behavioural change.

The POC is a deterministic discrete-event simulator of Northwind Goods' shared agent platform. Ten scenarios, each with a naive and a controlled arm on the same seed, arrivals, cases and clients:

#	Claim	Naive arm	Controlled arm
E1	Admission enforces capacity before the runtime	max work in system 5,791, goodput 9.3 %	128 (bound 128), goodput 45.5 %
E2	Tenant-aware controls beat global limits	support served in time 3.6 %	100 %, finance still completes (374.9 s)
E3	Concurrency is bounded independently of demand	max running 1,227, goodput 44.2 %	32, goodput 84.4 %
E4	Explicit, propagated budgets stop runaways	runaway refund 18,960.3 cu	150 cu; 4 legitimate workflows cut
E5	Routing optimises inside a contract	all-large 24.9 cu per success	routed 10 cu, 0 violations
E6	Context and caches need budgets and scope	34,716 tokens per query; cache leaks 5	5,982, required evidence missing 0; leaks 0
E7	Tool capacity is its own limit	237 503s, 2.8 attempts per call	0 503s; composed with fair scheduling, support 100 %
E8	A non-code change is a new release	1 image digest	7 release ids
E9	Invariant gates block before traffic	—	traffic to blocked candidates: 0 %
E10	Canaries read behaviour, not status codes	error rate 0 pp different	tool calls per workflow 88.9 % → ROLLBACK

The proof pack holds 11 experiments (the ten scenarios and the negative control) and 65 checks: 63 pass, 1 fails as a recorded limitation (the envelope cut legitimate work), and 1 is the negative control, which removes admission and must fail. Three results that are not in the headline matter as much as the ones that are. A tool gateway on its own protected the payments API and starved support (5.3 % served), because the wait moved into the runtime's slots. A canary that compared raw window averages rolled back a clean release. And under the declared success table, the all-small arm was cheaper per success than routing; it also broke the eligibility contract on 516 model calls. Section 32 covers all three.

2 • Where this sits in the series

This is the operations chapter of Production AI Engineering. It is written against the architecture the earlier notes built and adds no new layer.

Earlier note	What it already owns	What this note adds
F2 · layered platform	six layers, with control planes across them; “cost / quotas” named as one of them	where load lands in each layer and which layer owns which control
F3 · headless AI	the capability layer; model routing and cost architecture as design	amplification measured; routing inside a contract; cost per successful outcome
T4 · AI control plane	model profiles, budgets and quotas, a canary <code>{version, percent}</code> , “rollback is one pointer write”, kill switches	the full behavioural release manifest; canaries judged on behaviour; rollback that cannot undo committed effects
R1 + R2 · evals, observability, reliability	change → offline evals → gate → canary; execution certainty; a retry budget that does not model load; “ No load . Retry storms, backoff, budgets and circuit breakers were outside the experiment.”	the load R1 + R2 left out: admission, fairness, backpressure, retry storms, budgets under contention; promotion on operational evidence
P1 · capstone	four planes, layers A–H, runtime budgets, a model gateway, a kill switch; scale and operational maturity named as future work	that future work, on P1's architecture, unchanged
C1 · multi-agent and A2A	when an agent boundary earns its coordination cost; a workflow with agents against a coordinator with specialists; A2A as an integration boundary	what that coordination costs under load: envelopes and telemetry that propagate down the agent tree
T1, T2, T3, T5, T6	identity, policy, approval, lineage, security	only where scale or lifecycle touches them: fallback policy, tenant-scoped caching, approval policy as a release artifact, release invariants

C1 (Do You Actually Need Multiple Agents?) is written; this note links to it where coordination meets load and claims none of its results as its own. **S2** (enterprise knowledge and RAG) is written too, and referenced here by concept only. This note uses their vocabulary where load and change touch them: coordinator, sub-agent, delegation, retrieval, index.

The vocabulary is the series'. The runtime path is P1's eight layers: A experience and integration, B request and identity boundary, C orchestration and durable execution, D agent runtime, E context and memory, F model services, G tool and action platform, H enterprise systems (F2's six layers plus P1's request boundary and the systems of record). Across them run P1's four planes: **control** (what may exist and run: registries, versions, policies, budgets, kill switches), **runtime** (what is happening now), **enforcement** (may this happen, now: identity, policy, approval, budget) and **evidence** (what happened). The control plane follows T4's rule, “define centrally, enforce where agents run”, and stays out of the hot path.

3 · The incident: Black Friday on a shared platform

Northwind Goods (the series' fictional store, seed 4917) runs one shared agent platform for several internal tenants. Two matter here. **Support** is customer-facing and interactive: order status,

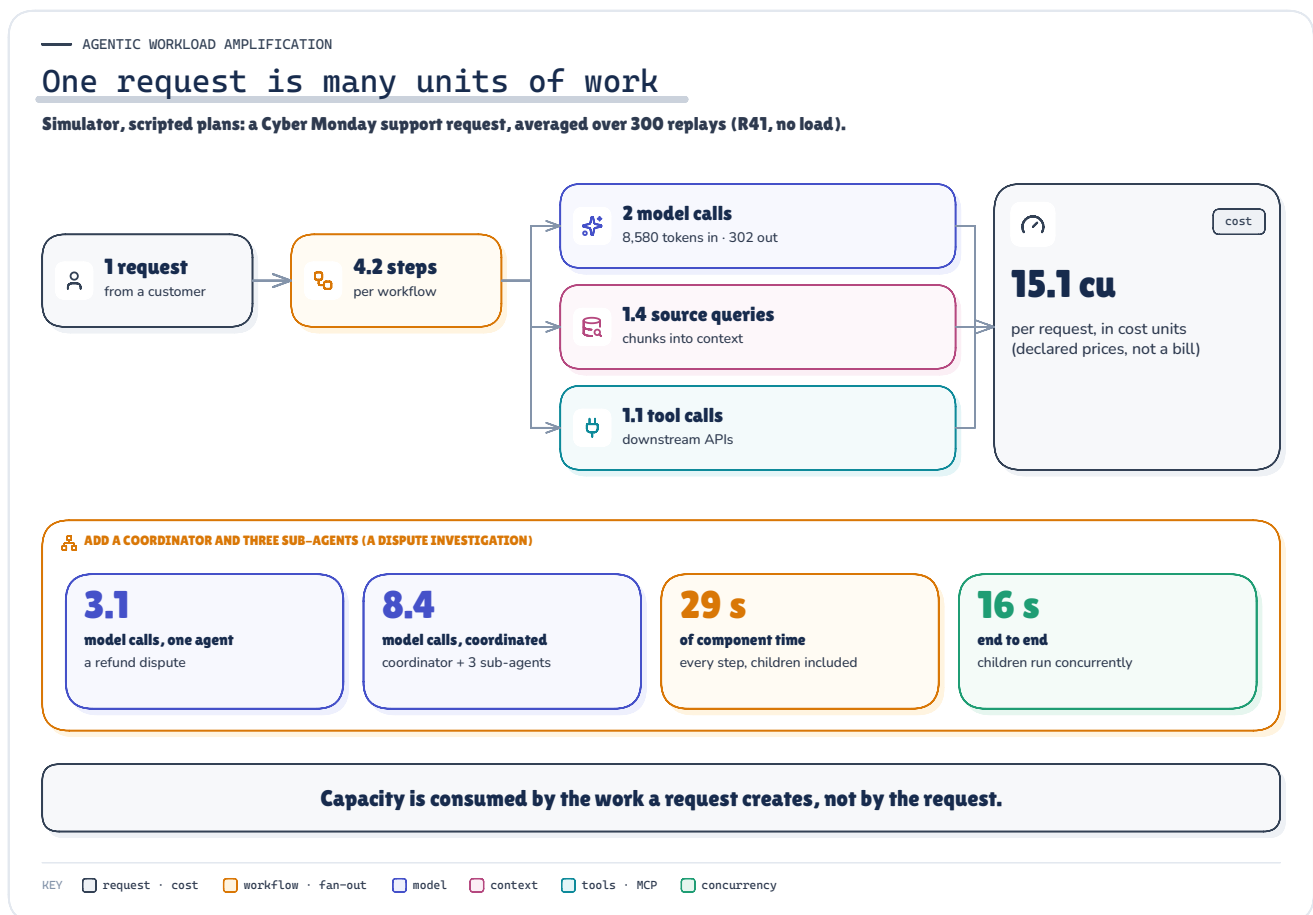
delivery changes, refund disputes, product questions. **Finance** is a batch tenant: after Black Friday it reconciles payments, thousands of checks at once.

On Black Friday, 27 November 2026, support traffic surges to a multiple of a normal day. The double capture at 10:15 that earlier notes followed is background here; that was a reliability problem and R1 + R2 settled it. This note's problem is operational. In the same hour, Finance starts a bulk reconciliation run on the same platform, and the noisy neighbour turns out to be your own finance team. Requests become workflows. Workflows become model calls, retrieval queries and tool calls. The queue grows. The EU model deployment throttles. The payment processor's status API, far smaller than the agent runtime, starts refusing connections. Clients time out and retry. Cost climbs with no customer served.

Then, on the Monday, someone ships a change: a tool description, one sentence. No code.

4 • Scaling an agent is a workload-amplification problem

Request rate alone does not describe the load on an agent platform: what matters is the rate times the work each request creates. One external request becomes a workflow; the workflow becomes steps; the steps become model calls with their input and output tokens, retrieval queries and the chunks they bring into context, tool calls, and, with a coordinator, sub-agents that each do the same. SRE made the point for ordinary services: “Different queries can have vastly different resource requirements”, so “A better solution is to measure capacity directly in available resources”^[1]. For agents the spread is wider, because the amount of work a request creates is decided at run time.



MEASURED Figure 1. One request is many units of work: a Cyber Monday support request under R41, averaged over the replayed sample, and a dispute investigation with a coordinator and three sub-agents. Measured: 300 Cyber Monday requests replayed under R41 (E8) and the coordinators of E4 · run 2026-10-08-recorded

In the POC's replayed Cyber Monday sample (300 requests, release R41, no load), one support request created 4.2 workflow steps, 2 model calls carrying 8,580 input and 302 output tokens, 1.4 source queries and 1.1 tool calls: 4.5 operations per request, before any retry. A dispute investigation run by a coordinator and three sub-agents made 8.4 model calls against 3.1 for a single-agent refund dispute. That is why OpenTelemetry's GenAI conventions now define per-invocation counts, `gen_ai.invoke_agent.inference_calls` and `gen_ai.invoke_agent.tool_calls` [23]. It is also why Anthropic reported, for its own research product, that “agents typically use about 4× more tokens than chat interactions, and multi-agent systems use about 15× more tokens than chats” [45]. That is one product's data, not a constant.

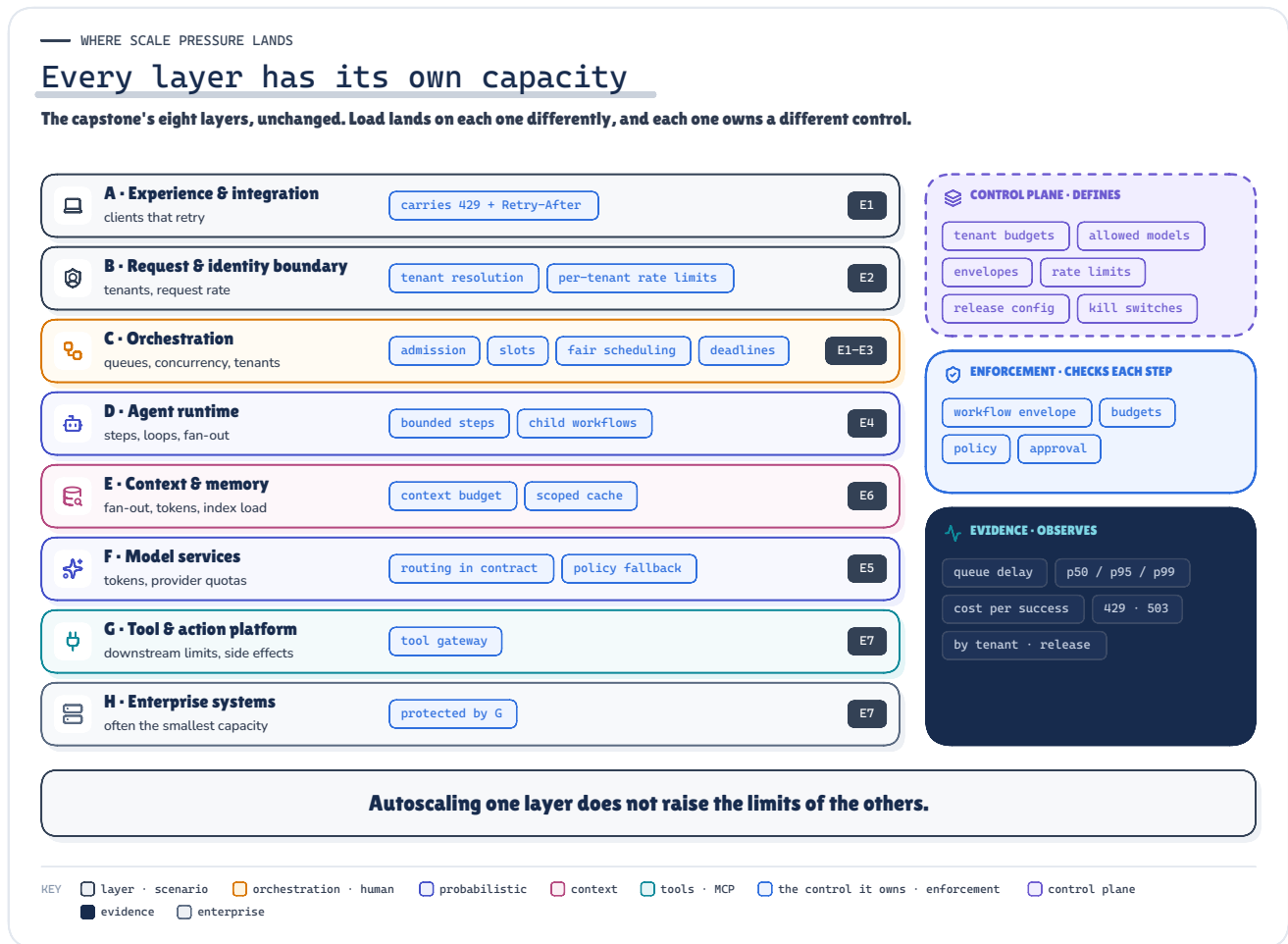
A useful mental model, **our synthesis** rather than a formula from a source:

$$\text{effective platform load} = \text{external demand} \times \text{internal work per workflow}$$

(requests/s) (steps, model calls, tokens, retrievals, tool calls, children)

Each factor is a distribution, not a number, and the second one is set by the agent at run time. Little's Law, “the average number of items in a queuing system ... equals the average arrival rate ... multiplied by the average waiting time” ^[9], then says how much concurrency a given demand requires. AWS's example: 100 messages a second at 100 ms need about 10 threads, and “If the latency suddenly spiked to 10 seconds, it would suddenly use 1,000 threads” ^[6]. An agent step's latency depends on output length and on how many steps the agent decides to take. So a platform sized on last week's mean latency can fall over on Monday without any change in request rate.

5 · Where the pressure lands



ARCHITECTURE Figure 2. Where scale pressure lands: the capstone's eight layers, the load each one feels, the control each one owns, and the scenario that tests it. The control plane defines; the enforcement plane checks each step; the layers enforce; the evidence plane observes.

Architecture: the capstone's layers (P1) with the control each owns; scenario tags point to the experiments

Each layer of P1's runtime path has its own capacity and its own failure mode under load:

- **A · Experience and integration.** Clients that retry. The layer itself decides nothing (P1); its job under load is to carry the platform's capacity error and `Retry-After` back to a client that honours them (E1).
- **B · Request and identity boundary.** Tenant resolution and per-tenant rate limits, which P1 already places here. It is the first place a tenant's demand can be shaped.
- **C · Orchestration.** Admission against capacity, queues, concurrency, tenants, deadlines. This is where demand meets capacity (E1, E2, E3).
- **D · Agent runtime.** Steps, loops, fan-out. Without an envelope, the agent decides how much work a request costs (E4). The envelope is checked by the **enforcement plane** before each step:

P1's budgets are an enforcement primitive the agent cannot negotiate.

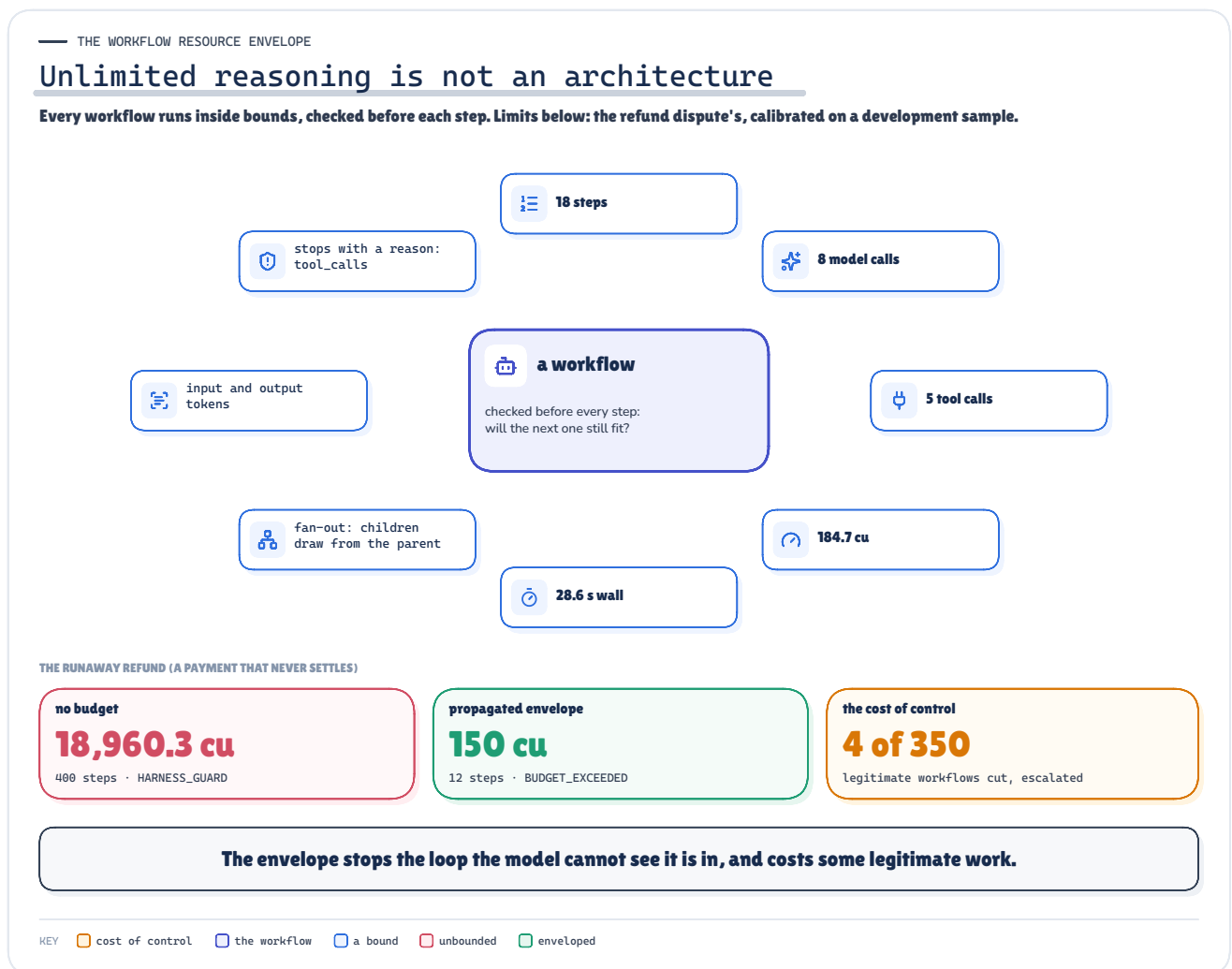
- **E • Context and memory.** Retrieval fan-out, context tokens, index load, and caches that can leak or go stale (E6).
- **F • Model services.** Tokens and provider quotas in several dimensions: requests, input tokens and output tokens per minute ^[17], or tokens per minute against an allocatable pool ^[19] (E5).
- **G • Tool and action.** Downstream limits, often the smallest in the system, and side effects (E7).
- **H • Enterprise systems.** The systems of record, protected through G.

Autoscaling the runtime raises none of the other limits. SRE: “An increase in traffic will have consequences down the stack. Backend services, such as databases, need to absorb any additional load your servers might create”, and in a dependency outage “Autoscaler will scale up the jobs, causing more and more traffic to get stuck” ^[3]. The Kubernetes HPA scales “to match observed metrics such as average CPU utilization” ^[12]. It does not raise a provider's token-per-minute limit or a payment processor's connection pool.

6 • Give every workflow a resource envelope

Unlimited reasoning is not a production architecture. An agent that decides at run time how many steps to take has to run inside bounds the platform enforces. Anthropic's guidance already includes “stopping conditions (such as a maximum number of iterations) to maintain control” ^[44]. The **workflow resource envelope**, our synthesis, generalises that single limit into a set of bounds:

```
Workflow resource envelope (checked before every step)
├─ max wall-clock time
├─ max reasoning steps
├─ max model calls
├─ max input / output tokens
├─ max estimated cost
├─ max retrieval operations
├─ max tool calls and side-effect attempts
└─ max sub-agent fan-out (children draw from the parent's envelope)
```



ARCHITECTURE **MEASURED** Figure 3. The workflow resource envelope, with the refund dispute's limits as the POC calibrated them, and what happened to a runaway refund without and with it.

Architecture + measured: the refund dispute's calibrated envelope and the runaway outcomes (E4) · run 2026-10-08-recorded

The envelope is one of five kinds of limit, and they are easy to confuse:

Limit	Who sets it	What it protects	Where it is enforced
Tenant budget	the control plane	other tenants and the business: spend and share per tenant	rate limits at the request boundary (B); admission and scheduling (C)
Workflow envelope	the control plane, per workflow class	the platform from one runaway workflow	the enforcement plane, before each step of the agent runtime (D)
Request budget	the caller or the capability contract	the caller's own latency and cost expectations	the request boundary and the runtime (B, D)
Provider quota	the model provider	the provider's shared capacity; to you, an external constraint to operate within, never elastic capacity	the model gateway (F); 429 with retry-after ^[17]
Downstream quota	the owner of the system of record	that system	the tool gateway (G)

A provider quota protects the provider's tenants, not yours: “Rate limits are defined at the organization level and at the project level, not user level” ^[18]. Fairness between your own customers needs your own quotas ^[5].

Two design rules follow from the POC.

- **Propagate the envelope to children.** In E4, per-agent budgets bounded every sub-agent and still let the coordinator's total exceed its envelope (11 model calls against a limit of 10). With the propagated envelope, where children draw from the parent's ledger, it stayed within (9).
- **An envelope has false positives.** It is checked before a step, and a step that starts inside it can finish over it by at most its own size. Calibrated on a development sample, it stopped 4 legitimate workflows of 350 in the blind sample, 4 of them workflows that had re-run once after a wrong answer (reasons recorded: delivery-change (steps); dispute-investigation (wall_ms)). Budgets must allow for legitimate retries, and a workflow stopped on its envelope must end in an explicit state: escalated, with its reason, never failed silently.

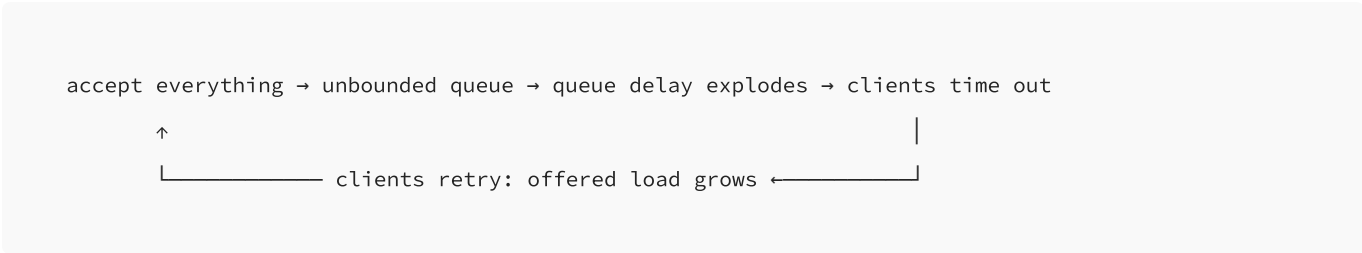
The envelope ties back to reliability and security. A runaway loop is a reliability failure that burns money, and a prompt-injected agent asked to “keep trying” is a security failure that burns money. The envelope bounds both without having to diagnose either.

7 • Admission control comes before execution

An overloaded platform should not accept unlimited work and hope autoscaling catches up. **A queue is not capacity.** SRE: “Queued requests consume memory and increase latency”, so it is better to keep queues short and reject “requests early when it can’t sustain the rate of incoming requests” ^[2]. AWS describes the failure mode a queue creates when arrivals exceed processing:

work “completed too late for the results to be useful, essentially causing the availability hit that queueing was meant to guard against” [6].

The feedback loop is worse for interactive agents, because their clients retry:



Retries are “selfish”, and “When failures are caused by overload, retries that increase load can make matters significantly worse” [7]. The useful measure is **goodput**, “the subset of the throughput that is handled without errors and with low enough latency for the client to make use of the response” [4].

E1 sends Black Friday support traffic at 20 requests per second for 120 seconds against 32 slots, with the same clients in both arms: they give up after 20 s and retry at once, up to 2 times. With **open admission** (an unbounded queue), work in system reached 5,791, the p99 queue delay was 686,675 sim-ms, clients submitted 2.8 attempts per request, and goodput was 9.3 %: 97.4 % of the cost went to attempts that finished after their client had given up. With **bounded admission** (work in system at most slots + queue bound = 128, and above it an explicit capacity error with `Retry-After: 4 s`), work in system never exceeded 128, the p99 queue delay was 14,027 sim-ms, and goodput was 45.5 %.

E1 · Black Friday surge: 20.0 requests/s for 120 s against 32 slots

	open admission	bounded admission
support requests	2640	2640
attempts (retries incl.)	7430	4564
max work in system	5791	128
max queued	5759	96
p99 queue delay, ms	686675	14027
refused explicitly	0	1405
served while waiting	245	1200
goodput %	9.3	45.5
cu on abandoned work %	97.4	57.9

bound: work in system <= slots + queue bound = 128; over it -> 429 + Retry-After 4 s

E1 · surge in run `2026-10-08-recorded`, as `agentops explain E1-surge` prints it from the recorded files.

The admission controller does not create capacity. It refused 53.2 % of requests explicitly, and that is the price. The surge exceeded what 32 slots can serve, and the only choice was between refusing early, with a clear error the client can act on, and accepting everything while serving almost nobody in time. Even inside the bound, 57.9 % of the cost still went to attempts that outlived their

client. Long refund disputes waited in the queue and then ran past the client's timeout. Admission bounds the work; it does not bound waste. Deadlines do (§8).

Admission checks, in a production platform, more than queue length: tenant quota (a token bucket per tenant, “the burst capacity because these tokens can be consumed instantly” ^[5]); estimated cost, admitted on an estimate and debited on the actual, as AWS describes for operations whose “cost ... is not always known up front” ^[5]; and priority class, so that the platform sheds the lowest class first ^[1].

8 • Orchestration owns concurrency, queues and backpressure

The orchestrator is not just the thing that calls the agent. It owns resource coordination: bounded concurrency, bounded queues, backpressure, fair scheduling, workflow limits, priority, cancellation and deadlines.

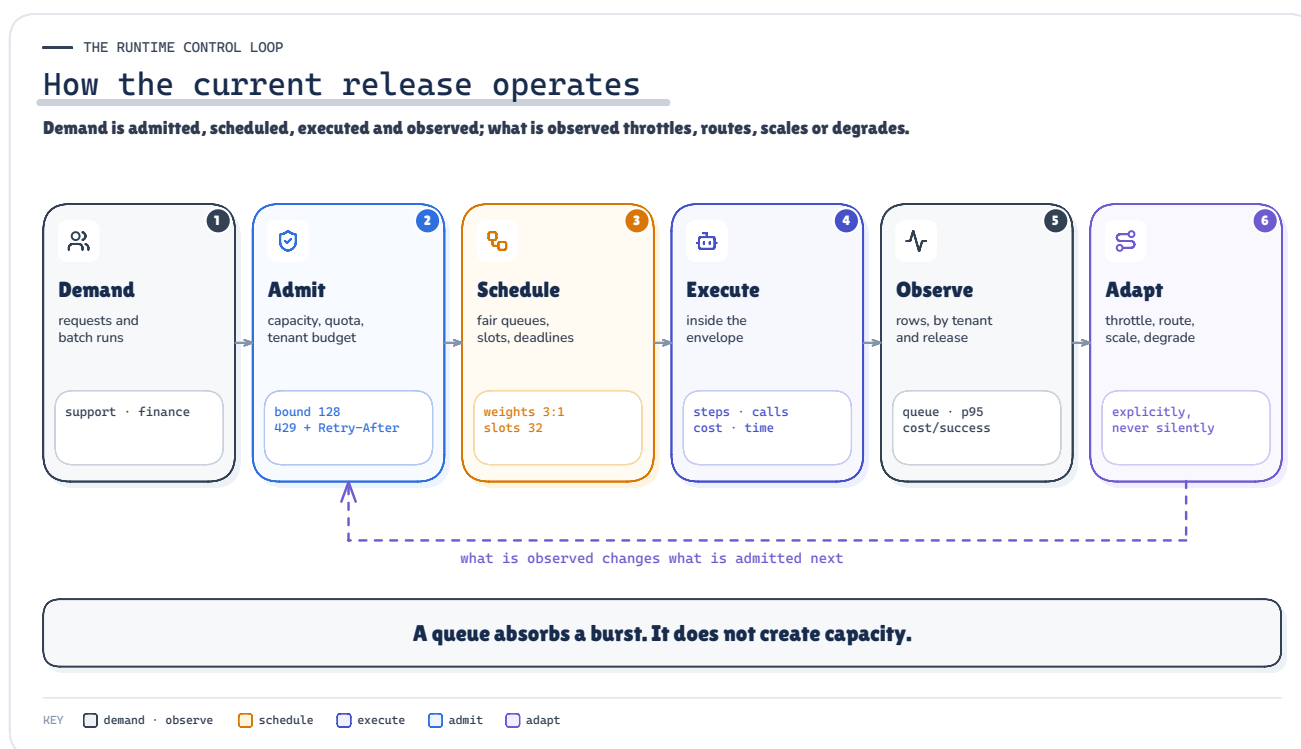
Bounded concurrency, independently of demand. Netflix's argument: “Instead of thinking in terms of RPS, we should be thinking in terms of concurrent requests” ^[10]. Envoy's: “It's nearly always better to fail quickly and apply back pressure downstream as soon as possible” ^[11]. **E3** runs an overload at about one and a half times capacity. With **unbounded concurrency** every admitted request started at once, and running workflows peaked at 1,227. The shared resources behind the runtime are modelled as processor sharing with capacity 32, so every workflow slowed together: p95 latency 251,507 sim-ms, goodput 44.2 %, and 95.2 % of the cost spent on attempts nobody was waiting for. With **slots, a bounded queue and deadlines**, running attempts never exceeded 32, goodput was 84.4 %, and wasted cost was 7.8 %.

Deadlines. “You don't get credit for late assignments with RPCs”; a multi-stage server “should check the deadline left at each stage before attempting to perform any more work” ^[2]. The orchestrator propagates the client's deadline into the workflow. At dequeue, an attempt that cannot finish before its client gives up is dropped (36 in E3). At run time it is cancelled cooperatively at a step boundary, never in the middle of a side effect. No attempt started after its client's deadline (0).

Fair scheduling and the noisy neighbour. Global limits protect the platform from overload. They do not protect one tenant from another. AWS enforces quotas “at a per-tenant or per-workload granularity”, so “the unplanned portion of that workload is rejected, and the other workloads continue operating with predictable performance” ^[5]. Netflix notes a system with live and batch traffic “may want to give live traffic 100% of the limit during heavy load and is OK with starving batch traffic” ^[10]. **E2** runs support at 5 requests per second while Finance submits 3,000 reconciliation checks in ten seconds.

	one global FIFO	weighted fair scheduling (3:1) + finance bulkhead (16)
support served within the timeout	3.6 %	100 %
support queue delay, p99	351,027 sim-ms	499 sim-ms
finance running at once, max	32	16
finance batch makespan	168.1 s	374.9 s (deadline 900 s)

Fairness is not free: the batch took 2.2 times as long. It was work-conserving, though. Finance used whatever support did not need, and every item completed within its deadline. The patterns are per-tenant concurrency caps (bulkheads), weighted fair queuing (deficit round robin in the POC), workload classes with priorities, and per-tenant queues, so that “By the time work is queued up in a shared queue, it’s hard to isolate one workload from another” ^[6] never applies.



ARCHITECTURE Figure 4. The runtime control loop: demand is admitted, scheduled, executed inside the envelope and observed; what is observed throttles, routes, scales or degrades the next admission.

Architecture: our synthesis; the stage values are the POC's declared configuration

9 · Model services own routing economics

Routing is a cost decision made inside a contract. The principle is **use the least expensive execution path that continues to satisfy the required quality and safety contract**, not “always use the cheapest model”. The research framing is a trade-off: “the choice of which model to use often

involves a trade-off between performance and cost”, and routers between a strong and a weak model can reduce cost “without compromising the quality of responses” on the benchmarks studied [24]. Cascades are the other family [25]. Anthropic's pattern is “Routing easy/common questions to smaller, cost-efficient models ... and hard/unusual questions to more capable models” [44]. Managed routers exist, and they are themselves a moving release surface: one version “is updated in place as new models become available”, which “could affect the overall performance of the model and costs” [27].

The contract has more terms than quality: **capability** (is this model eligible for this task?), **data policy and residency** (may this model see payment data?), **latency**, **availability** and **cost**. In the POC the contract is explicit (`config/models.toml`): each task has a minimum capability tier and a data class; payment data may only go to the EU private deployments. **The contract and the success table are declared assumptions.** In production, which model is eligible for which task is established by evals (R1 + R2), not by a table. E5 tests routing *within* the contract. It says nothing about the quality of any real model.

E5 runs 800 workflows under four routing modes, with the EU private deployment throttled for 30 s:

	all-large	all-small	routed (eligible, policy-aware fallback)	routed, any fallback
success	96.4 %	98 %	99.1 %	100 %
first answer wrong (caught)	30	81	33	33
cost per request	24 cu	2.5 cu	9.9 cu	10.3 cu
cost per success	24.9 cu	2.6 cu	10 cu	10.3 cu
calls below the capability tier	0	516	0	0
calls breaking the data policy	0	0	0	18
deferred during the throttle	29	0	7	0

Inside the contract, routing cost 59.9 % less per successful workflow than sending everything to the large models, with no violation. **All-small was cheapest per request and, under this declared table, also per success.** It broke the capability contract on 516 calls and got 81 first answers wrong. The simulator assumes a validator that catches every wrong answer and re-runs the workflow once. Section 13 returns to what that assumption hides. Routed-any-fallback had the best availability of all, 100 %, because it sent payment data to a global model 18 times while the EU deployment was throttled.

Fallback must not cross a boundary. AWS's definition is “Use a different mechanism to achieve the same result”, and its warning is that fallback strategies “often make the outage worse” and have “latent bugs that show up only when an unlikely set of coincidences occur” [8]. A fallback model is a different mechanism. It may not be eligible for the task, may not be allowed the data, may not hold the residency the tenant was promised, and may not be covered by the evals the primary passed. Policy-aware fallback picks only from the eligible set and **defers explicitly** when the set is exhausted (7 deferrals in E5). It never degrades silently into a policy breach.

Provider throttling. Providers enforce several dimensions at once and differ in how they count. Anthropic counts uncached input tokens and actual output tokens ^[17]. OpenAI reserves “the maximum of max_tokens and the estimated number of tokens” ^[18]. Azure's estimate “includes max_tokens” and “isn't the same as the token count used for billing” ^[19]. A 429 carries a retry-after. The rules for clients are already written: “Treat this value as a minimum ... add a small random delay”, and “disable SDK retries or account for them in those limits so nested retry loops don't multiply requests” ^[18]. Retry at one layer, inside a budget: Google's rule is “a per-request retry budget of up to three attempts”, and “requests should only be retried at the layer immediately above the layer that is rejecting them” ^[1]. In an agent, the SDK retry, the framework retry and the model's re-plan are three nested loops unless the model gateway owns retries.

10 • Context and knowledge have operational cost

Context is not free because it is already stored. Every retrieved chunk is retrieval work, index load, input tokens, latency and cost on every model call that carries it. And past a point, more context is worse, not just more expensive: “Context, therefore, must be treated as a finite resource with diminishing marginal returns” ^[46]. S1 separated state, memory, knowledge and context. S2 owns retrieval quality. This note owns the operational half: fan-out, token budgets, caching and freshness.

E6 assembles context for twelve fixture queries. Each query declares in the fixture the evidence ids an answer **requires**, so “required evidence preserved” is a mechanical check, not a relevance judgement. Naive retrieval (every source, top-k 20, full history) used 34,716 tokens per query. Bounded retrieval (sources routed by query type, the release's top-k, a token cap, history trimmed to three turns and a summary) used 5,982, 82.8 % fewer. It missed 0 of the 15 required ids. The fixture's relevance scores are frozen, not a retriever, so this shows the budget can keep declared evidence. It does not measure retrieval quality.

Caching, cautiously. A cache reduces repeated work and creates staleness, authorization leaks and wrong reuse. Provider-side prompt caches are isolated per organization or workspace, and “Cache hits require 100% identical prompt segments” ^[20]. They do not isolate your end customers inside one workspace. An application retrieval or response cache has to carry its own scope. E6 replays a 24-lookup sequence with a knowledge version change and a policy change:

cache key	hits	served to another customer	served stale
query text only	15	5	5
tenant · principal scope · knowledge version · policy version · query	7	0	0

“Where is my order?” is the same text for every customer and a different answer for each. The scoped key treats personal questions as principal-scoped and shared ones (“What is your return

policy?”) as tenant-scoped, so it still hits where reuse is safe. A knowledge-index version or a policy version in the key makes a release invalidate the cache by construction. Tool-definition changes already do that on the provider side: “Modifying tool definitions (names, descriptions, parameters) invalidates the entire cache” [20].

11 · Tools have a different scaling limit

Tools call systems whose limits are usually far smaller than the model runtime's: rate limits, connection pools, API quotas, database capacity, side-effect throughput. **E7** puts the payment processor's status API (capacity 8 concurrent calls, connection limit 16, 20 requests per second) behind a runtime with 64 slots, during a finance run of 1,500 items and support traffic.



MEASURED Figure 5. A control in one layer moves the queue to the next: direct calls, the tool gateway alone, and the gateway composed with fair scheduling, measured at the payments API and at the requests. Measured: E7's three arms, from the payments API's counters and the requests · run 2026-10-08-recorded

With **direct calls** and immediate retries, calls in flight reached the connection limit (16), the API returned 237 503s and 5,932 429s, every logical call cost 2.8 attempts, and 1,311 workflows failed on the tool. Only 195 of 1,500 finance items completed. With the **tool gateway** (per-tool concurrency equal to the API's capacity, a token bucket at its rate limit, a bounded priority wait, a circuit breaker), calls in flight never exceeded 8, the API returned 0 503s, and every finance item completed.

And support starved: 5.3 % served in time. The gateway did exactly what it was built to do. Finance workflows waited at the gateway while holding runtime slots, so order-status requests that never touch the payments API could not get a slot. The queue moved from the downstream into the runtime. **Composed** with E2's fair scheduling and the finance bulkhead, the API was still protected (0 503s) and support was served (100 %). Controls compose; none of them is sufficient alone. This was a development-run finding, and the composed arm was added before the freeze (`experiments/DEVIATIONS.md` , P3).

Operational pressure also makes R2's reliability rules more important. More retries mean more chances of a duplicated side effect, and “A timeout or failure doesn't necessarily mean that side effects haven't happened” ^[7]. E7's tool is read-only. Writes still need R2's operation-id idempotency and reconciliation; this note does not re-teach them.

12 • Multi-agent systems multiply operational pressure

Decomposition can multiply aggregate work, tokens, cost and downstream pressure. Wall-clock latency is different: it depends on the critical path and on how much runs in parallel. One user workflow becomes a coordinator, which becomes three sub-agents, each of which retrieves context and calls models and tools. In E4's dispute investigations (scripted plans, so the shapes are ours) the coordinated workflow made 8.4 model calls against 3.1 for the single-agent version. Its components added up to 29 s of work, while it took 16 s end to end because the children ran concurrently. Three consequences:

- **The envelope must propagate** across child workflows (\$6). A per-agent budget bounds each agent, not the request.
- **Observability must propagate** too. OpenTelemetry records each sub-agent's calls against its own invocation, “so that each inference call is counted exactly once across the call tree” ^[23]. Attributing that tree to one request and one release is our addition.
- **Coordination pathologies cost money.** A coordinator that re-delegates a failing child is a loop. With no budget, E4's coordinator ran until the simulator's guard: 205 model calls, 16,735.1 cu.

C1 (Do You Actually Need Multiple Agents?) owns coordination patterns and A2A. In its local deployment, a multi-agent system's extra cost went to coordinator reasoning, re-read context, duplicate tool calls and review rounds, with the A2A transport a small share. This note claims only the operational consequence: whatever the decomposition, its work has to be budgeted and attributed down the tree.

13 · Cost must be attributable

Token price is not workflow cost. A production workflow pays for model inference, embeddings, retrieval and the search infrastructure behind it, agent compute, orchestration, state persistence, tool and API charges, observability, downstream infrastructure, and the people who handle what the agent could not. An illustrative decomposition, **not an accounting standard**:

```
workflow cost = Σ model cost + Σ retrieval cost + Σ tool / API cost + platform overhead  
(+ human handling)
```

The POC prices every component in cost units on every telemetry row (`cost_cu: {model, retrieval, tool, platform, escalation}`). By its declared prices the model dominates: 99.1 % of a Cyber Monday request's cost. That share is a property of the simulator's price table, not a finding. In a real platform, retrieval infrastructure, orchestration compute, observability and human handling can be material, and the mechanism that matters is **attribution**. Cost is useful only when it can be cut by request, workflow, capability, agent, tenant, model, tool and **release**. OpenTelemetry's GenAI token counters are explicitly “a proxy for cost approximation” and carry no price ^[22]. Attribution has to come from your own row schema.

Cost per successful outcome is our synthesis. The nearest sourced practice is tracking “latency, token usage, cost per task, and error rates ... on a static bank of tasks” ^[41]. Cost per request rewards a cheap call that fails. Cost per successful outcome charges the failure, the retry and the escalation to the outcome that eventually succeeded:

```
cost per successful outcome = (cost of every attempt, re-run and escalation) / outcomes that  
succeeded
```

Examples: cost per resolved support case, per completed reconciliation, per approved refund. E5 shows both why the metric matters and where it stops. All-small cost 2.5 cu per request and 2.6 cu per success on machine cost. Adding the declared 60 cu for each escalation to a person, it was 3.8 cu. That is still below routed (10 cu). Under this table, all-small stays cheaper per success unless one escalation costs more than 364 cu. The table hides the real issue. The validator in the simulator catches every wrong answer; a real one does not. A wrong refund decision that reaches a customer costs far more than its tokens. That is why routing optimises **within** an eligibility contract established by evals, rather than chasing cost per success alone. The cheaper model may or may not create the cheaper workflow; the contract decides which cheaper models you are allowed to consider.

14 · Latency must be decomposed

End-to-end latency is not model latency. It is queue delay, orchestration overhead, retrieval, model inference, tool latency, approval waits, sub-agent execution and persistence, combined along the **critical path**:

$$\text{request latency} = \text{queue delay} + \text{orchestration overhead} + \text{critical execution path}$$

When steps run concurrently, adding every component's latency is wrong. The coordinated dispute investigation spent 29 s in its components and took 16 s end to end. Under surge, the largest term was not the model at all. In E1's bounded arm, successful requests spent 70.6 % of their end-to-end time queued:

successful requests, E1 bounded admission	p50	p95
queue delay	11,589	13,643
model time	1,776	10,548
retrieval time	128	256
tool time	150	450
orchestration overhead	100	175
end to end	14,296	23,374 (p99 25,321)

(sim-ms; percentiles are nearest-rank per component, so the rows do not add up to the end-to-end row.)

Measure distributions, not means: “A simple average can obscure these tail latencies, as well as changes in them” ^[14]. Fan-out makes tails dominate: “Variability in the latency distribution of individual components is magnified at the service level” ^[13]. That paper concerns parallel fan-out to stateless servers. Agent steps are mostly sequential, so their tails add along the path, and hedging a side-effecting tool call is not something the paper supports.

15 · Observability for operations

R1 + R2 built observability for diagnosis: what failed, where, and with what certainty. Operations reads the same evidence for two other purposes: runtime decisions (admit, route, throttle, degrade) and release decisions (promote, roll back). The metrics, at minimum:

Family	Metrics
Demand and admission	request rate; admitted / rejected rate by reason; retry rate; attempts per request
Queues and concurrency	queue depth and age ^[6] ; queue delay; active workflows; concurrency by tenant; saturation
Workflow	duration p50 / p95 / p99; steps; envelope exhaustion by dimension; outcome (success, escalated, deferred, failed)
Models	calls; input / output tokens; latency; throttles (429) and retry-after; fallback rate; route distribution
Retrieval	queries; chunks; context tokens; latency; cache hit rate (and scope)
Tools	calls; latency; errors; 429 / 503; attempts per logical call; in-flight vs capacity; breaker state
Cost	cost per request, per successful outcome ; by tenant, capability, model, tool and release
SLOs	goodput; served-undegraded ratio ^[15] ; budget burn rate ^[16]

A dashboard without dimensions hides the problem. E2's global metrics looked healthy in the naive arm, since the platform was busy and throughput was high, while support was being served in time for 3.6 % of requests. The fault was visible only by tenant. E10's error rate looked healthy (a delta of 0 pp) while tool calls per workflow had risen by 88.9 %. That fault was visible only by release. The dimensions that matter: tenant, workflow type, capability, agent, release id, model, provider and tool.

On naming: OpenTelemetry's GenAI conventions record `gen_ai.request.model`, `gen_ai.usage.input_tokens` and `gen_ai.usage.output_tokens` on inference spans, and the inference-duration metric is now `gen_ai.client.inference.duration` ^[21]. Token metrics moved in September 2026 from the `gen_ai.client.token.usage` histogram to `gen_ai.client.inference.usage.*` counters ^[22]. Everything there is at **Development** stability and was renamed weeks before this note; pin the generation you emit. No convention exists for tenant, workflow budget, routing decision or release id. Those are custom, privately namespaced attributes.

16 • Degradation should be intentional

When capacity is constrained, a platform degrades whether or not anyone designed it to. The choice is whether it does so on purpose. Options, roughly from least to most visible:

```
route eligible work to a smaller eligible model → reduce optional enrichment → bound
retrieval fan-out →
disable non-critical steps → defer asynchronous non-critical work → reduce optional sub-
agent fan-out →
shed low-priority work → return an explicit capacity error
```

Degradation cannot silently cross a quality, security or policy boundary. Smaller models only within the eligibility contract; no fallback outside the data policy (E5's routed arm deferred rather than cross it); no approval skipped because the approver queue is long. Approval capacity is itself an operational bottleneck: T3's human approvals are a queue with a service rate. A degraded mode is a product decision with its own SLI: SRE asks you to “measure the proportion of responses that were served in an undegraded state” ^[15]. It is also code that rarely runs, and “the code path you never use is the code path that (often) doesn’t work” ^[2]. Exercise it.

17 · The AI control plane decides; the runtime enforces

T4 drew the line, and this note keeps it: the control plane “defines, versions, distributes and governs”, and the runtime selects, meters and enforces. For scale and lifecycle the split is:

Control plane defines (versioned, signed, distributed)	Runtime and orchestration enforce (per request, per step)
tenant budgets and quotas; priority classes	admission, token buckets, per-tenant concurrency
allowed models; the eligibility contract; provider policy	routing, quota waits, policy-aware fallback
workflow envelopes per workflow class	the ledger, checked before every step
rate limits per tool and downstream	the tool gateway
release configuration: which manifest serves which share of traffic	the traffic split; release id on every row
guardrails for canaries; gate thresholds	the canary controller's per-window decision
feature flags; kill switches	immediate local enforcement

The control plane is not in the path of every token (T4). Overloading it with runtime responsibilities makes it a bottleneck and a single point of failure. It defines constraints; the layers that run the work enforce them.

18 · From scale to lifecycle

With the controls above, the current version can run safely under load. Production agents have a second problem: the “current version” does not stay current for long. SRE's experience is that “roughly 70% of outages are due to changes in a live system” ^[28], and Google's canarying chapter adds that “a majority of incidents are triggered by binary or configuration pushes” ^[31]. For an agent, most behavioural changes are not binary pushes at all.

19 · The behavioural change surface

Production behaviour can change because of:

```
prompt · model · model parameters · model routing · agent code · workflow · tool implementation
·
tool description · tool schema · retrieval configuration · embedding / index configuration ·
knowledge corpus · memory policy · authorization policy · approval policy · operational
configuration ·
eval suite
```

“We did not deploy application code” does not mean “production behaviour did not change.”

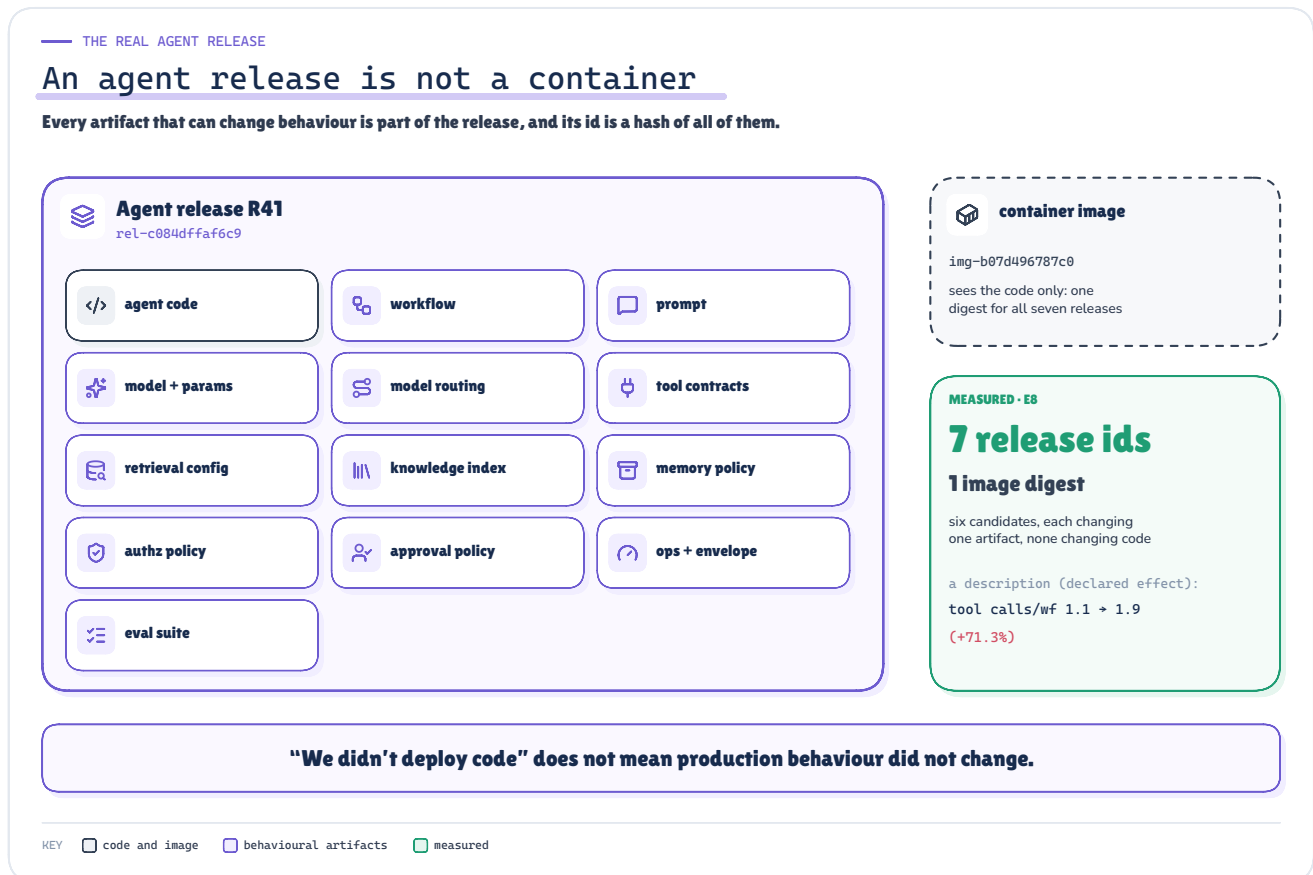
Sculley et al. named the problem for ML systems a decade ago: “CACE principle: Changing Anything Changes Everything”, which “applies not only to input signals, but also to hyper-parameters, learning settings, sampling methods ... and essentially every other possible tweak”; and “the number of lines of configuration can far exceed the number of lines of the traditional code” ^[38]. For agents the surface is wider:

- **A model alias is a dependency that changes.** The “same” LLM service “can change substantially in a relatively short amount of time” ^[39] (2023 snapshots of unpinned services; pin versions).
- **Tool descriptions shape behaviour, as prompts do.** They are different artifacts, but both are configuration the model reads. “Tool definitions and specifications should be given just as much prompt engineering attention as your overall prompts” ^[44]. MCP tools are “model-controlled” and the tool set “MAY change over time” ^[43], sometimes outside your deploy.
- **Small changes cascade.** “In agentic systems, minor changes cascade into large behavioral changes” ^[45].
- **Retrieval and corpus changes change answers,** and memory-policy changes change future decisions.

- **Policy changes change what is possible:** an authorization or approval policy is behaviour, too (T2, T3).

20 • The agent release unit

An agent release should be every artifact that can materially change behaviour, versioned as one unit:



ARCHITECTURE **MEASURED** Figure 6. The real agent release: every behavioural artifact is part of it, and its id is a hash of all of them. The container image sees only the code. Measured in E8: seven releases built from the same code.

Architecture + measured: the release manifest (agentops/release.py) and E8's seven releases · run 2026-10-08-recorded

The principle: if an artifact can materially change agent behaviour, its version must be attributable to the release that produced that behaviour. Not every platform needs this exact schema. The POC's manifest is the merged release configuration plus the content hash of the agent code, and the release id is a hash of the canonical manifest: computed, never assigned. Abridged, as the run recorded R41 (ops_poc/runs/2026-10-08-recorded/scenarios/E8/releases/R41.json):

```

{
  "release_id": "rel-c084dffaf6c9",
  "image_digest": "img-b07d496787c0",
  "manifest": {
    "agent_code": {"version": "support-agent 1.4.0", "sha256": "..."},
    "workflow": {"version": "support-flows 12"},
    "prompt": {"system": "support-system v7"},
    "model": {"temperature": 0.2, "max_output_tokens": 400},
    "routing": {"version": "routing v9", "strategy": "cheapest-eligible", "overrides": {}},
    "tools": {"orders.lookup": {"schema": "orders.lookup v3", "description": "Look up an order
by id. ..."}},
    "retrieval": {"top_k": 4, "max_sources": 2, "context_cap_tokens": 6000},
    "knowledge": {"index": "kb-2026-11-26", "embedding": "emb-v2"},
    "memory_policy": {"version": "memory v3"},
    "authz_policy": {"version": "policy v18", "delegated_refund_limit_eur": 100},
    "approval_policy": {"version": "approval v5", "refund_approval_above_eur": 100},
    "ops": {"envelope": "envelope v4", "envelope_enforced": true},
    "evals": {"suite": "ops-suite v1"}
  }
}

```

SRE's release engineering already snapshots “configuration files alongside their binaries” so that “we can use the build ID to reconstruct the configuration at a specific point in time” ^[29]. The workbook adds that rollback needs configuration to be **hermetic**: “Configuration that requires external resources that can change outside of its hermetic environment can be very hard to roll back” ^[30]. A prompt that reads a live knowledge index, or an unpinned model alias, is non-hermetic in exactly that sense. That is why the index version and the pinned model belong in the manifest. Telemetry can then answer **which behavioural release handled this request**, which is a better question than which Docker image did.

E8 builds R41 and six candidates, each changing exactly one artifact and none changing code: a prompt (R42-e), a tool description (R42-a), retrieval top-k, the knowledge index, an authorization policy version, a routing table.

E8 · the same code, seven releases

image digest (code): img-b07d496787c0 (identical for all 7)

release	release id	changed artifact	tools/wf	cu/wf
R41	rel-c084dffa6c9	(production)	1.117	15.08
R42-a	rel-ec085ba7c34e	tools.orders.lookup	1.913	15.2
R42-e	rel-0e00bddd9bfb	prompt	1.117	13.97
R42-kb	rel-3b3cce63002f	knowledge	1.117	15.08
R42-policy	rel-91e7d99815c9	authz_policy	1.117	15.08
R42-routing	rel-d540123fc17c	routing	1.117	18.5
R42-topk	rel-550efbb7626c	retrieval	1.117	20.52

E8 · release in run 2026-10-08-recorded, as agentops explain E8-release prints it from the recorded files.

The 7 releases shared one image digest and had 7 distinct release ids. Each diff named exactly one artifact. On the same 300 replayed Cyber Monday requests, 4 of the six candidates changed at least one behaviour metric. A routing override moved cost and a shorter prompt moved tokens. Two candidates (a knowledge index and a policy version) changed nothing *on this workload* and still got new ids, as they must: identity cannot depend on whether you noticed a behaviour change. The run wrote 58,573 telemetry rows, and 58,573 of them carry a release id.

R42-a's effect is declared, not measured. The scripted agent follows tool descriptions literally: a description that says “once per shipment” makes it call the tool per shipment. That stands in for the documented fact that models select tools from their descriptions ^[43] ^[44]. The size of the effect is a modelling assumption, and OPS-E8-C04 is an implementation check, not a hypothesis (`DEVIATIONS.md` , A2). The size also depends on the traffic mix. Tool calls per workflow rose 32.7 % on the quiet-week offline cases, 71.3 % on the replayed Cyber Monday sample, and 88.9 % in the canary's first window, where more orders ship in parts. What the run tests is everything downstream of the change: identity, attribution, the gate (E9) and the canary (E10).

21 · LLMOps is too small a name for what is being operated

LLMOps is a useful word, and it usually means the model and prompt lifecycle: versioning prompts, evaluating model upgrades, tracking token spend. The thing in production is an agent system, not an LLM call:

LLMOps	model · prompt
Agent lifecycle	model · prompt · workflow · context · memory · retrieval · knowledge · tools · policies · code · evals · operational configuration

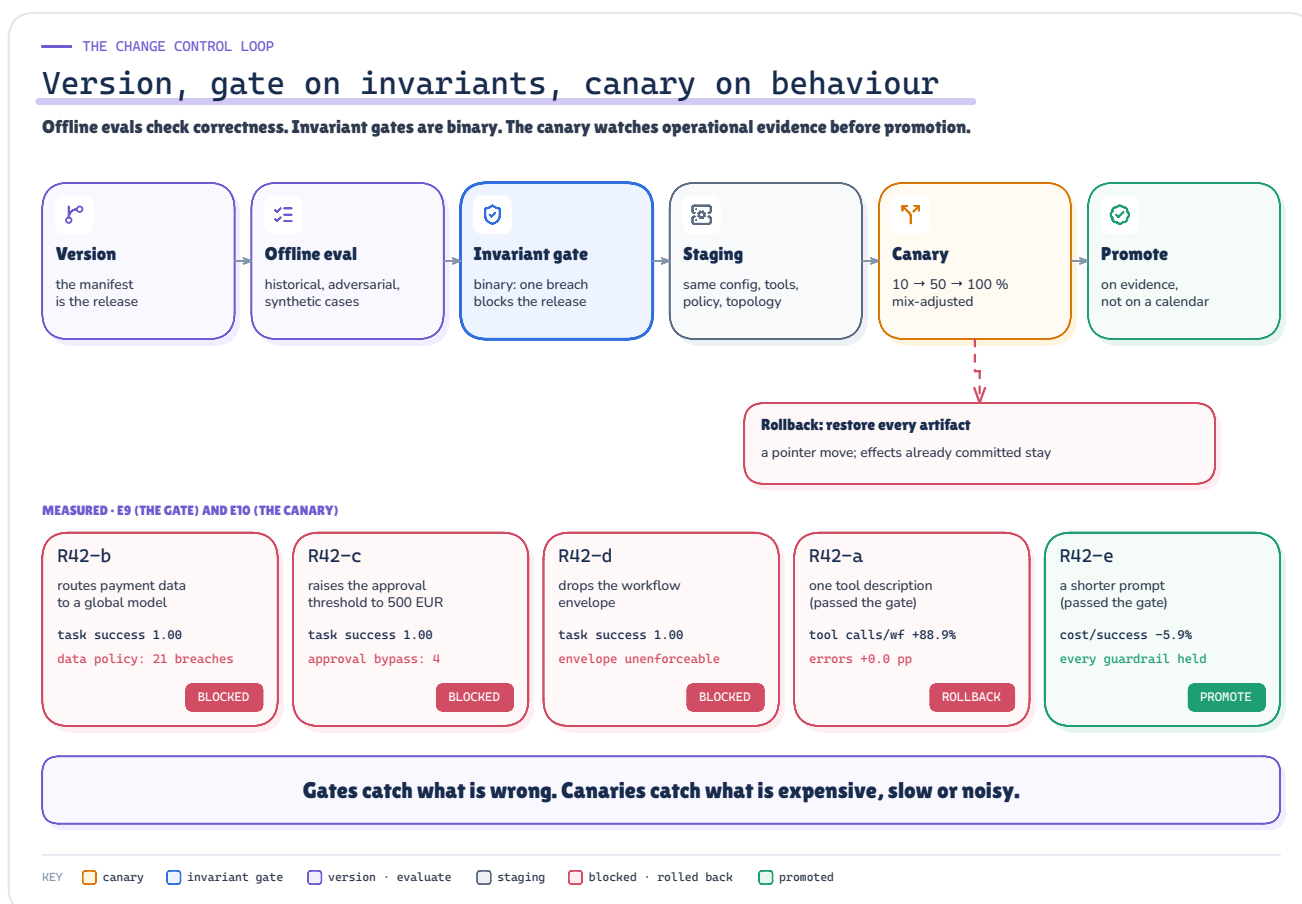
“We version prompts, so we have LLMOps” is true and is not enough. The lifecycle of an agent is the lifecycle of the release manifest.

22 · The change control loop

The release path:

version → offline eval → policy and invariant gates → staging → (safe shadow / replay) → canary
→ observe → promote

↳ roll back



ARCHITECTURE

MEASURED

Figure 7. The change control loop, and what it decided in the run: three candidates blocked by invariant gates before any traffic, one rolled back by the canary, one promoted. Architecture + measured: E9's gate decisions and E10's canary decisions · run 2026-10-08-recorded

Offline eval

Historical, synthetic and adversarial cases, checked for task quality, tool selection and arguments, policy compliance, security invariants, cost regression, latency regression and workflow behaviour. R1 + R2 built this discipline, and this note does not repeat it. The practice is settled: run evals “on each agent change and model upgrade as the first line of defense” ^[41], and avoid “Creating

eval datasets that don't faithfully reproduce production traffic patterns" ^[42]. The POC's offline suite has 40 cases: historical cases from a quiet week, adversarial cases (refunds above the delegated limit, payment data) and synthetic ones.

Policy and invariant gates

Some conditions are binary release blockers, not average quality metrics: a forbidden tool must never execute; tenant isolation must hold; a required approval cannot be bypassed; the workflow envelope must remain enforceable; a critical capability must stay above its eval threshold. **E9** submits five candidates:

E9 · offline suite (40 cases) and invariant gates

candidate	task ok	data	approval	envelope	cost Δ%	decision
R42-a	1.0	0	0	0	+0.7	PASSED
R42-b	1.0	21	0	0	-15.7	BLOCKED
R42-c	1.0	0	4	0	+0.0	BLOCKED
R42-d	1.0	0	0	1	+0.0	BLOCKED
R42-e	1.0	0	0	0	-7.2	PASSED

canary traffic granted to blocked candidates: 0 %

E9 · gate in run 2026-10-08-recorded , as agentops explain E9-gate prints it from the recorded files.

R42-b routes refund disputes to a cheaper global model (“save cost on disputes”). Its task success was 100 % against a threshold of 0.9, and its cost per success changed by -15.7 %. It also sent payment data to an ineligible profile 21 times. R42-c raises the approval threshold above the delegated refund limit: 4 refunds would have skipped a required approval. R42-d drops the workflow envelope. All three were blocked, each for its own invariant (an average cannot outvote one), and the registry refused each a canary weight: traffic to blocked candidates was 0 %. R42-a (a tool description) and R42-e (a shorter prompt) passed.

R42-a passing is the instructive part. The offline suite saw its tool calls per workflow rise by 32.7 %, and its cost per success moved 0.7 %, inside the gate's 10 % regression limit. Nothing in the gate said how many tool calls is too many, and the gate checked that the right tools were called. That is an operational property, and the canary owns it.

Staging

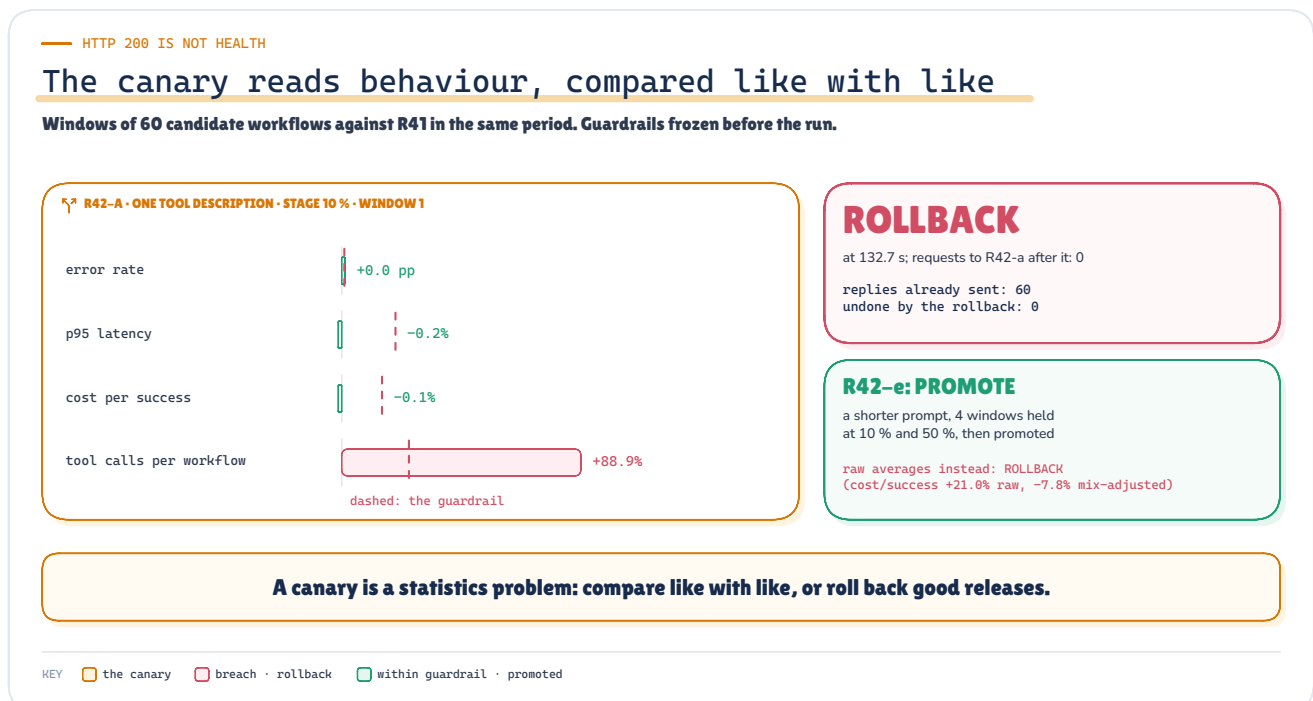
Staging should reproduce the configuration, tool contracts, policy, retrieval behaviour and workflow topology of production. It does not reproduce production traffic, its mix, or its scale. It catches integration errors, not distribution shifts.

Shadow and replay, where safe

Mirroring sends “a copy of live traffic to a mirrored service” as “fire and forget”, and “the responses are discarded” [37]. The response is discarded; the side effects are not. Flagger's rule is the one to keep: mirroring is for “requests that are idempotent or capable of being processed twice” [35]. An agent shadow run that calls real tools acts twice. Shadow evaluation of an agent therefore separates **read-only evaluation**, **simulated actions**, **recorded fixtures** and **sandboxed tools** from genuine production side effects. The POC's offline replays (E8, E9) run every case against simulated, read-only tools for that reason.

Canary

A canary is “a partial and time-limited deployment of a change in a service and its evaluation” [31]. It is compared “against an equivalent baseline, deployed at the same time” [36], on “the top few metrics ... (perhaps no more than a dozen)” [31]. For agents the metrics are behavioural, not only process health. A release can return HTTP 200 while it becomes more expensive, slower, less accurate, more tool-happy or less compliant. The POC's guardrails, frozen in the preregistration, compare candidate with baseline per window: cost per success at most +15 %, p95 at most +20 %, tool calls per workflow at most +25 %, error rate at most +1 pp, success at least -2 pp, and no policy violation.



MEASURED

RECORDED

Figure 8. HTTP 200 is not health: R42-a's first analysis window against R41, guardrails dashed, and the rollback record; R42-e promoted under the mix-adjusted comparison and rolled back under a raw one.

Measured + recorded: R42-a's first analysis window and the rollback record; R42-e under both comparisons (E10) · run 2026-10-08-recorded

E10 splits Cyber Monday traffic 10 / 50 / 100 % by a hash of the request id, in windows of 60 candidate workflows:

```
E10 · canary R42-a (rel-ec085ba7c34e) vs R41 (rel-c084dffaf6c9), 60 candidate workflows per window  
  
stage 10% window 1: error +0.0 pp · success +0.2 pp · p95 -0.2% · cost/success -0.1% · tool calls/wf +88.9%  
breaches: tool_calls_pct  
  
decision: ROLLBACK at 132.7 s · requests to the candidate afterwards: 0  
replies already sent by the candidate: 60 · undone by the rollback: 0
```

E10 · canary in run `2026-10-08-recorded`, as `agentops explain E10-canary` prints it from the recorded files.

R42-a passed the offline gate, kept its error rate (a 0 pp delta) and its cost per success (-0.1 %). Its extra tool calls are the scripted agent's declared behaviour (§20). What E10 measures is the canary's response to them. Its extra calls ran on the small model, so the money barely moved. Tool calls per workflow rose 88.9 %, which is downstream load on the orders API that another team pays for, and the canary rolled it back in its first window. R42-e, the shorter prompt, held for 4 windows at 10 % and 50 % and was promoted.

A canary is a statistics problem. Cost per workflow differs by more than an order of magnitude between an order-status check and a refund dispute, and a 60-workflow window can easily hold more refund disputes than the baseline did. The controller therefore compares candidate and baseline **per workflow type, weighted by the baseline's mix in the same period**. Kayenta asks the same question, “Are these two sets of numbers meaningfully different?”, with a statistical test and “at least 50 pieces of time series data per metric” ^[36]. Run against the clean R42-e with **raw window averages** instead, the same controller rolled it back (ROLLBACK): raw cost per success 21 % in the window that tripped it, mix-adjusted -7.8 %. A canary that rolls back good releases teaches teams to stop trusting it. Argo Rollouts' answer to an unclear signal is to pause rather than decide: “Inconclusive runs causes a rollout to become paused” ^[34]. AWS's is to bake: wait “for a specific number of data points” before judging ^[32].

Promote

Promotion happens on evidence: every guardrail held for the required windows at each stage, and the last step (100 %) is the promotion itself. After promotion, R42-e served 1,396 requests as production.

Roll back

Rollback is not `kubectl rollout undo`. Behaviour depends on the prompt, the model, the routing table, the tool contract, the policy, the corpus, the index, the memory configuration and the workflow. A rollback has to restore a *compatible set* of all of them, which is why the registry rolls

back to the previous release **id**, every artifact at once. T4 put it well: for a configuration pointer, rollback is one pointer write. This note adds where that stops.

- **Committed side effects are not rolled back.** R42-a's canary workflows sent 60 replies before the decision; the rollback undid 0. Rolling back configuration is not undoing external effects. Write-capable canaries need appropriately scoped traffic, sandboxed or simulated execution where possible, or reconciliation for effects already committed (R2).
- **State written in a new format may not roll back.** AWS: “After the new version writes some compressed data, rolling back isn’t an option” [33]. A memory record in a new schema, or a tool argument in a new contract, is the agent equivalent.
- **Index and corpus updates** may not be reversible if the old index is not retained. A rollback target is valid only if every artifact it names still exists.
- **Runs in flight** straddle versions. Anthropic deploys “rainbow deployments to avoid disrupting running agents” [45]. Pin the release id per run.

Not every agent change is trivially reversible, and the release process should say which ones are not before they ship.

23 · Release observability

Every trace should attribute execution to its behavioural release. Fields such as `trace_id`, `tenant`, `workflow`, `agent_release`, `prompt_version`, `model`, `routing_policy`, `tool_contract`, `knowledge_version` and `policy_version` are implementation-specific and, under OpenTelemetry, custom attributes. The capability that matters is correlating a behavioural change with an operational outcome:

release R41	p95 = X	cost/success = Y	tool calls/wf = Z	
release R42-a canary	p95 ≈ X	cost/success ≈ Y	tool calls/wf » Z	eval score unchanged →
roll back				

Promotion then becomes evidence-driven rather than calendar-driven. Every row the POC writes carries `release_id`, and E10's decisions are computed from those rows alone.

24 · The POC: what it is and what it is not

`ops_poc/` is a **deterministic agent-operations simulator** of Northwind Goods' shared platform, written for Python 3.12 using only the standard library. `uv` manages its environment and `pytest`

runs its tests; neither is part of the standard library or of the simulator. No model is called, nothing touches a network, and no API key is needed.

What it proves. That specific mechanisms change modelled resource use and behaviour under a declared workload: admission, fair scheduling, bounded concurrency with deadline-aware queues, workflow envelopes, routing inside an eligibility contract, context budgets with scoped caching, a tool gateway, a content-addressed release manifest, an invariant gate and a behavioural canary with rollback.

What it does not prove. Real provider latency, quotas or prices; real model quality; how much a prompt or tool description change moves a real model's behaviour (the scripted agent's response is declared); real tool limits; human approval capacity; the behaviour of shadow traffic; rollback of stateful artifacts; or that any organisation's bill or latency will move by the simulated amounts. Every number is a **simulation unit**: simulated milliseconds (sim-ms) and cost units (cu). A claim of the form “the control reduced modelled resource use under the defined workload” is defensible; “this architecture cuts LLM bills by X %” is not, and this note does not make it.

Class	What it covers
Implemented (real code, executed on every simulated request)	admission, deficit-round-robin fair scheduling, slots, deadline-aware dequeue and cooperative cancellation; the envelope ledger shared with children; routing, quota waits and policy-aware fallback; bounded retrieval and the scoped cache; the tool gateway (concurrency, token bucket, bounded priority wait, circuit breaker); release manifests, the registry, the gate and the canary controller
Simulated (a deterministic local substitute)	the clock; model providers (latency, quotas, 429); downstream APIs (capacity, 503, 429); clients (timeouts, retries); the agents themselves, scripted so the platform is the only variable
Declared (an assumption the article states)	the eligibility contract and success table; processor-sharing contention beyond 32 concurrent workflows; a validator that catches every wrong answer; the price table in cu
Injected	the Black Friday surge; the finance run; a payment that never settles; a re-delegating coordinator; a 30 s provider throttle; the candidate releases; the negative control
Architecture only	autoscaling, real providers, approval capacity, shadow traffic, stateful rollback

25 · Method: preregistration, run, evidence

The series' discipline (R1 + R2), unchanged:

- Preregistration.** ops_poc/experiments/preregistration.toml holds the question, what is held constant, every hypothesis with its threshold, and how each metric is computed. It was written before the scenario code and frozen with the configuration and fixtures (FROZEN.sha256) before the recorded run. At least one hypothesis per family could fail, and one did.

2. **Development run.** One development run (D1) was used to debug the simulator before the freeze. What it changed is logged in `experiments/DEVIATIONS.md` (P1–P6): two bugs, a third arm for E7, and the canary's mix adjustment, with the raw comparison kept as a recorded arm instead of being hidden. No threshold changed after D1.
3. **Recorded run.** `make record` runs every scenario once from the frozen inputs into `ops_poc/runs/2026-10-08-recorded/` : raw rows, requests, timelines, platform counters, release manifests, eval results and canary decisions. `facts.json` is then aggregated **from those files**, not from in-memory counters.
4. **Proof pack.** `proof/experiments.toml` declares 65 checks, each comparing a fact with a preregistered value or with the other arm. `proof/claims.toml` maps the claims to the checks. `tools/proof_pack.py` evaluates them (pae-proof/v1) into `evidence/runs/2026-10-08-recorded/` , and `results/claim-evidence.json` is the claim → scenario → mechanism → evidence → result mapping, generated.
5. **Replay.** `make replay` re-executes every scenario from source into a scratch copy and compares file by file: EXACT, 99 of 99 files byte-identical. Only wall-clock timings (`volatile/`) are excluded.
6. **Verification.** `make verify` recomputes `facts.json` from the raw files in a scratch copy, re-evaluates every check and claim, checks SHA256SUMS over the raw run, the replay record, the negative control, the preregistration hash, the printed facts and a secret scan.
7. **Negative control.** E1's controlled arm with the admission bound removed. Its check must fail, and it does (OPS-NC). A proof that cannot fail proves nothing.

26 · Implementation map

Module	P1 layer	What it does
<code>sim.py</code>	—	discrete-event engine: virtual clock, event heap ordered by (time, sequence), generator processes, slots with an ordered wait, token buckets with outage windows
<code>platform.py</code>	C · Orchestration	admission (open / bounded), FIFO or weighted fair scheduling with per-tenant caps, slots, deadline-aware dequeue, cancellation at the deadline; exact maxima at every event
<code>runtime.py</code>	D · Agent runtime	executes a plan step by step: cancellation, envelope and harness-guard checks before each step; retrieval, model and tool calls; children; the outcome model and one re-run before escalation; one telemetry row per attempt
<code>budget.py</code>	D	the envelope and the ledger (children draw from the parent's in the propagated mode)
<code>agent.py</code>	D (scripted)	the plans per workflow type, including the runaway loop and the re-delegating coordinator; follows tool descriptions literally (a declared behavioural model)
<code>router.py</code>	F · Model services	profiles, the eligibility contract, the four routing modes, provider quotas, policy-aware or unsafe fallback, the outcome probability
<code>retrieval.py</code>	E · Context and memory	bounded retrieval for load scenarios; the fixture assembler with required-evidence checks; the query-text and scoped caches
<code>tools.py</code>	G · Tool and action	the downstream overload model; the naive client; the tool gateway
<code>release.py</code>	Control plane	manifests, content-addressed release ids, the image digest, diffs, the registry
<code>scenarios.py</code>	—	the ten scenarios, the offline evaluator and gate, the canary controller, the negative control
<code>run.py</code> · <code>cli.py</code>	—	record, aggregate, explain, freeze, prereg-check, demo

27 · The ten scenario tests

`tests/test_scenarios.py` holds exactly ten tests, one per claim. Each re-runs its scenario from source and asserts the mechanism and the comparison; none reads the recorded run. `tests/test_units.py` adds unit tests of the engine, slots, token buckets, fair scheduling order, the ledger, release identity, the registry, cache scope, the eligibility contract, the gate and determinism. Result: 23 of 23 passed (10 of 10 scenario tests, 13 unit tests). A passing scenario test says the mechanism behaved as specified, not that its claim was supported. The preregistered checks decide that (\$29): of the 10 claims tested, 1 was qualified and the others were supported.

Test	Claim	Asserts
test_01	admission enforces capacity before the runtime	work in system $\leq$ bound; open queue grows past it; refusals happen before execution; retry amplification higher when open; goodput not lower
test_02	tenant-aware controls	support $\geq$ 99 % served in time; FIFO lower; finance $\leq$ bulkhead; every finance item completes before its deadline
test_03	bounded concurrency	running $\leq$ slots; unbounded $>$ capacity; nothing starts after its deadline; work is shed at dequeue; goodput higher
test_04	explicit, propagated budgets	no budget $\rightarrow$ harness guard; envelope $\rightarrow$ BUDGET_EXCEEDED within limits; coordinator within envelope; per-agent exceeds it
test_05	routing inside the contract	routed cheaper per success than all-large; zero routed violations; explicit deferral; any-fallback breaks data policy; all-small breaks capability
test_06	context budgets and cache scope	every declared required id present; context within the cap; fewer tokens; scoped cache never leaks or serves stale; query-text cache does both; scoped still hits
test_07	tool capacity governed independently	gateway arms $\leq$ capacity and no 503; direct calls reach the connection limit; more attempts per call; composed support $\geq$ direct
test_08	a non-code change is a new release	one image digest; distinct release ids; one artifact per diff; a description changes tool calls; every row has a release id
test_09	invariant gates before traffic	exactly R42-b, R42-c and R42-d blocked; no weight granted; R42-b's average met the threshold; R42-a and R42-e pass
test_10	behavioural canary and rollback	R42-a rolled back; error delta within 1 pp; no request reaches it afterwards; replies stay sent; R42-e promoted

28 · The telemetry row

One row per workflow attempt, written as JSON lines; every fact is aggregated from these rows:

```
request_id · attempt · tenant · workflow · release · release_id · arm · admission_result ·  
arrival_ms · start_ms · end_ms · queue_delay_ms · latency_ms · deadline_ms · deadline_met ·  
client_gave_up_ms · client_waiting ·  
steps · model_calls · model_routes{profile: n} · input_tokens · output_tokens ·  
tool_result_tokens ·  
cost_cu{model, retrieval, tool, platform, escalation} · retrieval_calls · retrieved_chunks ·  
context_tokens ·  
tool_calls · tool_attempts · tool_busy · throttles · fallbacks · capability_violations ·  
data_violations ·  
approvals_requested · children · workflow_attempts · time_ms{model, retrieval, tool, overhead,  
wait} ·  
result · budget_reason · rollback_reason
```

The run wrote 58,573 rows. Costs are in cu and times in sim-ms, so these rows carry no provider billing precision, by design.

29 • Results

Run 2026-10-08-recorded: 11 experiments (ten scenarios and the negative control), 65 checks, 63 pass, 1 recorded limitation, 1 expected failure (the negative control). Replay EXACT. The full per-scenario tables are in the run report (`results/operating-ai-agents-report`). The claim-by-claim proof is in the Evidence Check (`results/operating-ai-agents-evidence`).

Ten claims, ten scenarios, one recorded run

Each scenario ran a naive and a controlled arm on the same workload. Simulation units. Run 2026-10-08-recorded.

	measure	without the control	with it	
1 Admission	max work in system	5,791	128	✓
2 Tenant fairness	support served in time	3.6 %	100 %	✓
3 Bounded concurrency	goodput	44.2 %	84.4 %	✓
4 Resource envelope	runaway refund cost	18,960.3 cu	150 cu	QUALIFIED
5 Routing in contract	cost per success	24.9 cu	10 cu	✓
6 Context budget	tokens per query	34,716	5,982	✓
7 Tool gateway	503s at the payments API	237	0	✓
8 Release identity	ids for 7 releases (image / manifest)	1	7	✓
9 Invariant gate	traffic to blocked releases	no gate	0 %	✓
10 Behavioural canary	R42-a: errors / decision	+0.0 pp	ROLLBACK	✓

QUALIFIED: the envelope also cut 4 legitimate workflows of 350 (a limitation the run observed).

65 checks: 63 pass, 1 limitation, 1 negative control that failed as it must.

KEY ■ qualified ■ controlled

MEASURED

Figure 9. Ten claims, one recorded run: one line per scenario, the naive or baseline arm against the controlled one, and the check counts of the proof pack.

Measured: one line per claim, naive and controlled arms; check counts from the proof pack · run 2026-10-08-recorded

30 · The claim–evidence table

#	Production claim	Experiment	Evidence (run 2026-10-08-recorded)	Result
1	Capacity is enforced before unlimited work enters the runtime	E1 · Black Friday surge, open vs bounded admission	work in system 5,791 → 128 (bound 128); attempts per request 2.8 → 1.7; goodput 9.3 % → 45.5 %	SUPPORTED
2	Multi-tenant platforms need tenant-aware controls	E2 · support + finance batch, FIFO vs fair + bulkhead	support in time 3.6 % → 100 %; finance max 16; makespan 168.1 → 374.9 s	SUPPORTED
3	Concurrency is bounded independently of demand	E3 · overload, unbounded vs slots + deadlines	running max 1,227 → 32; goodput 44.2 % → 84.4 %; wasted cost 95.2 % → 7.8 %	SUPPORTED
4	Production agents need explicit budgets, propagated to children	E4 · runaway refund, coordinator, blind sample	runaway 18,960.3 → 150 cu; coordinator model calls per-agent 11 vs propagated 9 (limit 10); legitimate cut 4	QUALIFIED
5	Routing optimises within a quality/policy contract	E5 · four routing modes, EU throttle	cost per success 24.9 → 10 cu; routed violations 0; any-fallback 18	SUPPORTED
6	Context and retrieval need resource controls	E6 · fixture queries, cache sequence	tokens 34,716 → 5,982; required ids missing 0; cache leaks 5 → 0, stale 5 → 0	SUPPORTED
7	Tool capacity is governed independently of model capacity	E7 · payments API behind a 64-slot runtime	in flight 16 → 8; 503s 237 → 0; attempts per call 2.8 → 1; composed support 100 %	SUPPORTED
8	Non-code changes create a distinguishable release	E8 · R41 + six one-artifact candidates	image digests 1; release ids 7; rows with a release id 58,573 of 58,573 (R42-a's behaviour change is declared)	SUPPORTED
9	Releases pass machine-checkable gates before traffic	E9 · five candidates, 40 offline cases	R42-b task success 100 % yet blocked; traffic to blocked 0 %	SUPPORTED
10	Releases need behavioural canaries and rollback on operational evidence	E10 · 10 / 50 / 100 % canary	R42-a: error delta 0 pp, tool calls 88.9 % → ROLLBACK; R42-e → PROMOTE	SUPPORTED
N C	The proof can fail	E1 controlled, admission removed	work in system 5,791 > 128	NEGATIVE CONTROL

31 · The cost of control

None of these controls is free, and the run records what each one cost:

- **Admission refused work.** 53.2 % of Black Friday requests got an explicit capacity error. The platform could not serve them in time either way; it told them early.
- **Fairness slowed the batch.** Finance's reconciliation took 2.2 times as long (374.9 s against 168.1 s), still inside its deadline.
- **Deadlines and bounds shed work.** 36 attempts were dropped at dequeue in E3 and 132 requests refused, in exchange for goodput rising from 44.2 % to 84.4 %.
- **The envelope cut legitimate work.** 4 of 350 blind-sample workflows were stopped and escalated.
- **Routing deferred work.** 7 refund disputes were deferred during the throttle rather than sent to an ineligible model.
- **Gates and canaries cost evaluation compute and time:** 40 offline cases per candidate, and a canary that reaches promotion only after 4 windows (320.6 simulated seconds).
- **The release manifest, the telemetry row and the scoped cache are engineering complexity:** more fields, more keys, more things to version.

The argument is not that controls are free. It is that uncontrolled amplification and uncontrolled behavioural change become far more expensive and riskier at production scale. In E1, open admission spent 97.4 % of its cost on work nobody waited for, while bounded admission refused some requests explicitly.

32 · What failed, or was qualified

Reported with the same prominence as what held:

- **The envelope's no-false-positive hypothesis failed** (OPS-E4-C05, LIMITATION OBSERVED). Calibrated as the development sample's maximum $\times 1.25$, it stopped 4 legitimate blind-sample workflows (delivery-change (steps); dispute-investigation (wall_ms)), every one of which had re-run after a wrong answer (4). A workflow that legitimately retries consumes twice; an envelope calibrated on first attempts does not allow for it. Remedies: calibrate on retried workflows, allow an explicit retry multiplier, or let the envelope escalate rather than fail. The claim is QUALIFIED.
- **The tool gateway alone starved support** (5.3 % served; OPS-E7-C06, observed in development and confirmed in the recorded run). Protecting a downstream moves the queue upstream. Only the composed arm protected both.
- **A raw-average canary rolled back a clean release** (OPS-E10-C07). The mix-adjusted controller is a design choice the run justifies. It is not a statistical test: windows of 60 can still mislead, and a production controller should add a significance test or an inconclusive state that pauses.
- **All-small was cheaper per success than routing** under the declared table, with escalations at 60 cu and a perfect validator. The result depends on assumptions the simulation states and real systems violate: that every wrong answer is caught, and that a wrong answer costs only its tokens. The claim is about the contract, not about cost per success alone.
- **E1's bounded admission still wasted 57.9 % of its cost** on attempts that outlived their client: long refund disputes queued and then ran past the timeout. Admission bounds the work in the system; deadlines (E3) bound the waste. This was observed, not preregistered.
- **The cost composition** (99.1 % model) is the price table's, not a result.

33 · How the evidence maps to the article

Each statement in the editions that rests on the POC names its scenario. Each scenario's checks are in `proof/experiments.toml`, and each claim's classification is in `proof/claims.toml`. `results/claim-evidence.json` lists every claim, its scenario, its mechanism, every check with its expected and observed value and the raw files it reads, and its result. `make verify` rebuilds all of it from the raw run. No measured number in either edition, the README or a figure is typed by hand. Each is

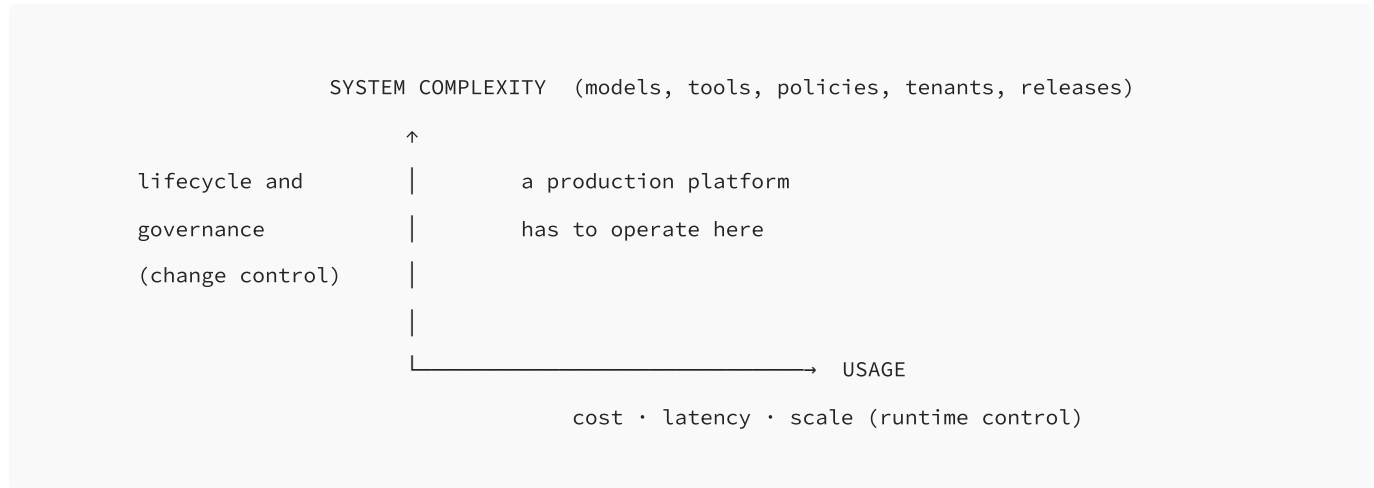
substituted from `ops_poc/runs/2026-10-08-recorded/facts.json` or `docs/derived-facts.json`, and `docs/evidence-uses.json` records every use.

34 · Anti-patterns

Anti-pattern	Why it is incomplete or dangerous
“Just autoscale it.”	Autoscaling adds runtime capacity, not provider quota, downstream connections, tenant fairness or budget. In a dependency outage it adds stuck work ^[3] .
“Just increase the context window.”	Context is retrieval work, tokens, latency and cost on every call, with diminishing returns ^[46] . E6: bounded retrieval kept every required id with 82.8 % fewer tokens.
“Just use the cheapest model.”	Cheapest is not eligible. E5's all-small broke the capability contract on 516 calls. Optimise inside the contract.
“Retry harder when the provider throttles.”	Retries are load. Honour retry-after, retry at one layer, inside a budget ^[18] ^[1] . E7's direct calls cost 2.8 attempts per logical call.
“Put everything in one global queue.”	A global queue cannot isolate tenants. E2: support in time 3.6 % under one FIFO.
“Queue depth is capacity.”	A queue absorbs a burst; it does not add capacity. Measure age, not just depth ^[6] . E1's open queue reached 5,759.
“Token cost is total cost.”	Retries, failures, escalations, retrieval, tools, orchestration and people are cost too. Attribute cost per successful outcome.
“Average latency looks fine.”	Tails and queue delay hide in means ^[14] . Under surge, queueing was 70.6 % of a successful request's time.
“We didn't deploy code, so production didn't change.”	E8: one image digest, 7 release ids, and a tool description changed behaviour.
“We version prompts, so we have LLMOps.”	The release is the whole manifest: routing, tools, retrieval, knowledge, policies, envelope, evals.
“The canary returned HTTP 200, so it is healthy.”	R42-a: error delta 0 pp, tool calls 88.9 %.
“Rollback means deploy the previous container.”	Behaviour lives in the manifest, and committed effects stay committed: 60 replies sent, 0 undone.
“Cache the response for everyone.”	E6's query-text cache served 5 personal answers to the wrong customer and 5 stale entries.
“Let the agent keep reasoning until it figures it out.”	E4's runaway refund ran to the harness guard: 18,960.3 cu. The envelope stopped it at 150 cu.

35 · Two axes: usage and complexity

At low scale a team can tolerate expensive model choices, large context windows, weak budgets, manual release checks, manual rollback and coarse observability. As usage and platform complexity both grow, each of those stops working:



Usage pushes along the horizontal axis. More requests, tenants and batch runs turn amplification, queueing, provider quotas and downstream limits into the binding constraints, and that is the runtime loop's territory. Complexity pushes along the vertical one. More models, prompts, tools, policies, indexes and releases turn every change into a possible behaviour change, and that is the change loop's territory. A platform can be high on one axis and low on the other: a single internal agent with heavy traffic, or a low-volume agent with fifty tools and weekly releases. A production platform ends up high on both, and the two loops have to share one evidence plane, because the same row that tells the runtime loop a tenant is over budget tells the change loop which release spent it.

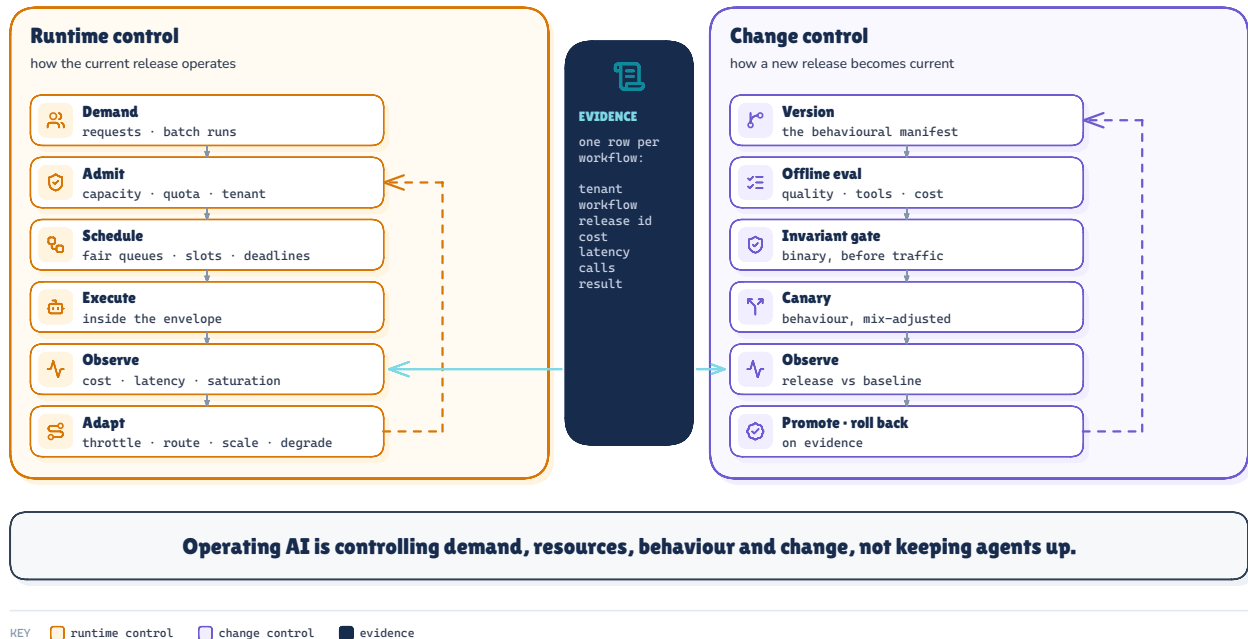
36 · Who owns what

The architecture is P1's. Scale assigns each existing layer its operational responsibilities:

Request boundary (B)	tenant resolution · per-tenant rate limits
Orchestration (C)	admission against capacity · queues · concurrency · fairness · backpressure · deadlines
Agent runtime (D)	cancellation at step boundaries · bounded reasoning · child workflows
Enforcement plane approval (T2, T3)	the workflow envelope checked before every step · budgets · policy and
Model services (F)	routing inside the contract · token budgets · provider quotas · fallback · cost/latency trade-offs
Context & knowledge (E)	context size · retrieval fan-out · scoped caching · index pressure · freshness
Tool & action (G)	rate limits · connection pools · downstream quotas · idempotency · side-effect throughput
AI control plane	tenant budgets · allowed models · usage policy · release config · feature flags · kill switches
Observability	cost per success · p50/p95/p99 · queue delay · model / retrieval / tool latency · saturation · SL0s

Two control loops, one evidence plane

The runtime loop governs demand and resources for the current release. The change loop governs which release is current.



ARCHITECTURE Figure 10. Two control loops, one evidence plane: the runtime loop governs demand and resources for the current release; the change loop governs which release is current. Both read the same rows.

Architecture: our synthesis (the two control loops and the shared evidence plane)

37 · Operational runbook

Signal	First question	Owning control	Typical action
queue age rising	arrivals above service rate, or service slower?	admission, slots	tighten admission; shed lowest priority; check provider latency
one tenant's p95 rising while global looks fine	is another tenant holding the slots?	fair scheduling, bulkheads	lower the noisy tenant's cap; check its batch
429s from a provider	which dimension (requests, input or output tokens)?	model gateway	honour retry-after; route within the eligible set; defer
downstream 503s / breaker open	is the gateway's concurrency above the downstream's capacity?	tool gateway	lower per-tool concurrency; back off; never retry writes blindly (R2)
envelope exhaustion rising for one workflow class	runaway, or legitimate retries?	envelope	inspect <code>budget_reason</code> ; recalibrate on retried workflows; escalate
cost per success rising for one release	more calls, more tokens, more retries or more escalations?	canary, release	compare the release's rows with the baseline, mix-adjusted
tool calls per workflow rising after a change with no code	which artifact changed?	release manifest	diff the manifests; roll back the release id
cache hit rate jumping after a release	did the key lose its scope or version?	context	check that tenant, principal and versions are in the key

38 · Production checklist

- [] Every workflow class has an envelope (steps, model calls, tool calls, tokens, cost, wall time, fan-out), enforced before each step, propagated to children, and ending in an explicit state with a reason.
- [] Admission bounds work in system and returns an explicit capacity error with retry-after; clients honour it.
- [] Concurrency is bounded per runtime and per tenant; queues are bounded and their age is measured; deadlines propagate and are checked at dequeue.
- [] Interactive and batch tenants are isolated (weights, bulkheads, priorities).
- [] Routing chooses from an eligibility contract established by evals; fallback stays inside it and defers rather than crossing it.
- [] Retries happen at one layer, inside a budget, honouring retry-after.
- [] Retrieval has a fan-out and token budget; caches are keyed by tenant, principal scope and knowledge and policy versions.
- [] Each downstream has a tool gateway sized to its capacity, composed with tenant-aware scheduling.

- [] Every row carries tenant, workflow, release id, model, tool, cost and outcome; dashboards cut by them.
- [] Cost is attributed per successful outcome, by tenant, capability and release.
- [] Degraded modes are designed, have an SLI, and never cross quality, security or policy boundaries.
- [] The release unit is the behavioural manifest, its id is content-addressed, and the model and router versions are pinned.
- [] Offline evals and binary invariant gates run before any traffic; blocked releases cannot receive weight.
- [] Shadow runs never perform real side effects.
- [] Canaries compare like with like on behavioural guardrails, and pause when inconclusive.
- [] Rollback targets a release id whose artifacts all still exist; irreversible changes are named before they ship.

39 • Limitations

This is a simulation. Its providers, tools, clients, prices and agents are deterministic substitutes, and its quality model is a declared table. It shows that specific mechanisms behave as claimed under a declared workload. It does not show how much any real platform will save, how real providers throttle, how real models route or fail, how human approval queues behave, or how a stateful rollback goes. The envelope's calibration, the canary's window size, the contention model and the client model are design choices of this POC. Different, defensible choices would move every number, and the mechanisms would still hold. Each scenario ran once; the run is deterministic, so a rerun reproduces it exactly, and that says nothing about variance in production.

40 • Conclusion: operating the platform the series built

The series started with an agent that had five hundred tools and no idea which one should run (F1). Then it built what has to exist around the reasoning:

Foundation	F1 tool sprawl · F2 layered platform · F3 headless AI
State & knowledge	S1 memory, context and state S2 knowledge and RAG
Authority & trust	T1 identity · T2 policy · T3 human approval · T4 control plane · T5 lineage · T6 security
Reliability	R1 + R2 evals, observability and recovery
Coordination	C1 multi-agent and A2A
Operations	O1 + O2 cost, latency, scale and lifecycle
	↓
	Production AI architecture (P1)

P1 assembled it and named scale and operational maturity as future work. This note is that work, on P1's architecture, unchanged:



ARCHITECTURE Figure 11. The platform, and what governs it: the capstone's eight layers, unchanged, with the four concerns that cut across them. Security protects what the system can see and do; evals, observability and reliability establish whether it is behaving correctly; multi-agent coordination (C1) decides when reasoning components should be separate agents and how they cooperate; scale and lifecycle decide whether it can sustain usage and evolve safely.

Architecture: the capstone's runtime path with the series' cross-cutting concerns

At production scale, cost and latency become architectural constraints, and every agent change becomes an operational event.

A production agent platform is not a model wrapped in an API. It is a governed execution system that has to control what agents know, what they can do, how they fail, how they coordinate, how much work they consume, and how their behaviour changes over time. Each of those has an owner in the architecture, and none of them is the model.

References

Every source was fetched on 2026-10-08 and is quoted in `research/sources.md` with the passage it supports and what it does *not* support. Model prices were deliberately not collected. Terms marked as our synthesis in the text (the workflow resource envelope, the behavioural release manifest, the two control loops, cost per successful outcome) are not defined by any source.

[1] Handling Overload — Site Reliability Engineering (Google, O'Reilly 2016), ch. 21 (Alejandro Forero Cuervo). sre.google/sre-book/handling-overload/

[2] Addressing Cascading Failures — Site Reliability Engineering (Google), ch. 22 (Mike Ulrich). sre.google/sre-book/addressing-cascading-failures/

[3] Managing Load — The Site Reliability Workbook (Google, O'Reilly 2018), ch. 11 (Cooper Bethea, Gráinne Sheerin, Jennifer Mace, Ruth King, with Gary Luo and Gary O'Connor). sre.google/workbook/managing-load/

[4] Using load shedding to avoid overload — Amazon Builders' Library (byline shown as "David", Sr. Principal Engineer; Builder Center republication 2026-06-12). aws.amazon.com/builders-library/using-load-shedding-to-avoid-overload/

[5] Fairness in multi-tenant systems — Amazon Builders' Library (byline shown as "David"; Builder Center republication 2026-06-12). aws.amazon.com/builders-library/fairness-in-multi-tenant-systems/

[6] Avoiding insurmountable queue backlogs — Amazon Builders' Library (byline shown as "David"; Builder Center republication 2026-06-12). aws.amazon.com/builders-library/avoiding-insurmountable-queue-backlogs/

[7] Timeouts, retries, and backoff with jitter — Amazon Builders' Library (byline shown as "Marc"; Builder Center republication 2026-06-12). aws.amazon.com/builders-library/timeouts-retries-and-

backoff-with-jitter/

[8] Avoiding fallback in distributed systems — Amazon Builders' Library (Jacob Gabrielson; Builder Center republication 2026-06-12). aws.amazon.com/builders-library/avoiding-fallback-in-distributed-systems/

[9] Little's Law as Viewed on Its 50th Anniversary — John D. C. Little, Operations Research 59(3):536–549 (May–June 2011), DOI 10.1287/opre.1110.0940; course-hosted copy (publisher page returned HTTP 403 to `curl`). people.cs.umass.edu/~emery/classes/cmpsci691st/readings/OS/Littles-Law-50-Years-Later.pdf

[10] concurrency-limits README — Netflix, GitHub [Netflix/concurrency-limits](https://github.com/Netflix/concurrency-limits) (README last changed at commit `b4e2167`, 2025-12-18). github.com/Netflix/concurrency-limits

[11] Circuit breaking; Adaptive Concurrency filter — Envoy proxy documentation (pages labelled "envoy 1.40.0-dev-2d188b"). www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/circuit_breaking

[12] Horizontal Pod Autoscaling — Kubernetes documentation (page last modified 2026-08-03). kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

[13] The Tail at Scale — Jeffrey Dean and Luiz André Barroso, Communications of the ACM 56(2):74–80, February 2013. research.google/pubs/the-tail-at-scale/

[14] Service Level Objectives — Site Reliability Engineering (Google), ch. 4 (Chris Jones, John Wilkes, Niall Murphy, with Cody Smith). sre.google/sre-book/service-level-objectives/

[15] Implementing SLOs — The Site Reliability Workbook (Google), ch. 2 (Steven Thurgood, David Ferguson, with Alex Hidalgo, Betsy Beyer). sre.google/workbook/implementing-slos/

[16] Alerting on SLOs — The Site Reliability Workbook (Google), ch. 5 (Steven Thurgood et al.). sre.google/workbook/alerting-on-slos/

[17] Rate limits — Claude Platform documentation (Anthropic). docs.claude.com/en/api/rate-limits

[18] Rate limits — OpenAI API documentation. platform.openai.com/docs/guides/rate-limits

[19] Manage Azure OpenAI in Microsoft Foundry Models quota (classic) — Microsoft Learn (ms.date 2026-05-04). learn.microsoft.com/en-us/azure/ai-foundry/openai/how-to/quota

[20] Prompt caching — Claude Platform documentation (Anthropic).

docs.claude.com/en/docs/build-with-claude/prompt-caching

[21] Semantic conventions for generative AI client inference — OpenTelemetry `semantic-`

`conventions-genai`, `docs/gen-ai/client-inference.md`, commit `06ec68e` (2026-10-07), no tagged release.
github.com/open-telemetry/semantic-conventions-genai/blob/06ec68e722c45a7218e23ea1bc1339fe

[22] Semantic conventions for generative AI inference token metrics — OpenTelemetry `semantic-`

`conventions-genai`, `docs/gen-ai/gen-ai-token-metrics.md`, commit `06ec68e` (2026-10-07).
github.com/open-telemetry/semantic-conventions-genai/blob/06ec68e722c45a7218e23ea1bc1339fe

[23] Semantic conventions for generative AI metrics: agent metrics — OpenTelemetry `semantic-`

`conventions-genai`, `docs/gen-ai/gen-ai-metrics.md`, commit `06ec68e` (2026-10-07). github.com/open-telemetry/semantic-conventions-genai/blob/06ec68e722c45a7218e23ea1bc1339fe

[24] RouteLLM: Learning to Route LLMs with Preference Data — Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, Ion Stoica, arXiv:2406.18665v4 (rev. 23 Feb 2025). arxiv.org/abs/2406.18665

[25] FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance — Lingjiao Chen, Matei Zaharia, James Zou, arXiv:2305.05176v1 (9 May 2023). arxiv.org/abs/2305.05176

[26] Understanding intelligent prompt routing in Amazon Bedrock — Amazon Bedrock User Guide.

docs.aws.amazon.com/bedrock/latest/userguide/prompt-routing.html

[27] Model router for Microsoft Foundry concepts — Microsoft Learn (ms.date 2026-09-01).

learn.microsoft.com/en-us/azure/ai-foundry/openai/concepts/model-router

[28] Introduction: Change Management — Site Reliability Engineering (Google), ch. 1 (Benjamin Treynor Sloss). sre.google/sre-book/introduction/

[29] Release Engineering — Site Reliability Engineering (Google), ch. 8 (Dinah McNutt).

sre.google/sre-book/release-engineering/

[30] Configuration Design and Best Practices — The Site Reliability Workbook (Google), ch. 14 (Štěpán Davidovič, with Niall Richard Murphy, Christophe Kalt, Betsy Beyer).

sre.google/workbook/configuration-design/

[31] Canarying Releases — The Site Reliability Workbook (Google), ch. 16 (Alec Warner, Štěpán Davidovič, with Alex Hidalgo, Betsy Beyer, Kyle Smith, Matt Duftler).

sre.google/workbook/canarying-releases/

[32] Automating safe, hands-off deployments — Amazon Builders' Library (Clare Liguori; Builder Center republication 2026-06-12). aws.amazon.com/builders-library/automating-safe-hands-off-deployments/

[33] Ensuring rollback safety during deployments — Amazon Builders' Library (Sandeep Pokkunuri; Builder Center republication 2026-06-12). aws.amazon.com/builders-library/ensuring-rollback-safety-during-deployments/

[34] Analysis & Progressive Delivery — Argo Rollouts documentation (stable). argo-rollouts.readthedocs.io/en/stable/features/analysis/

[35] Deployment Strategies — Flagger documentation. docs.flagger.app/usage/deployment-strategies

[36] Automated Canary Analysis (Kayenta) — Spinnaker documentation: "Canary Overview", "How canary judgment works", "Best practices for configuring canary". spinnaker.io/docs/guides/user/canary/canary-overview/

[37] Mirroring — Istio documentation (Traffic Management task). istio.io/latest/docs/tasks/traffic-management/mirroring/

[38] Hidden Technical Debt in Machine Learning Systems — D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, Dan Dennison (Google), NeurIPS 2015. proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.

[39] How is ChatGPT's behavior changing over time? — Lingjiao Chen, Matei Zaharia, James Zou, arXiv:2307.09009v3 (rev. 31 Oct 2023). arxiv.org/abs/2307.09009

[40] Define success criteria and build evaluations — Claude Platform documentation (Anthropic). docs.claude.com/en/docs/test-and-evaluate/develop-tests

[41] Demystifying evals for AI agents — Anthropic Engineering (Mikaela Grace, Jeremy Hadfield, Rodrigo Olivares, Jiri De Jonghe), published 9 Jan 2026. www.anthropic.com/engineering/demystifying-evals-for-ai-agents

[42] Evaluation best practices — OpenAI API documentation. platform.openai.com/docs/guides/evaluation-best-practices

[43] Tools — Model Context Protocol specification, revision 2026-07-28 (</specification/latest> resolves here). modelcontextprotocol.io/specification/2026-07-28/server/tools

[44] Building effective agents — Anthropic Engineering (Erik S., Barry Zhang), published 19 Dec 2024. www.anthropic.com/engineering/building-effective-agents

[45] How we built our multi-agent research system — Anthropic Engineering (Jeremy Hadfield, Barry Zhang, Kenneth Lien, Florian Scholz, Jeremy Fox, Daniel Ford), published 13 Jun 2025. www.anthropic.com/engineering/multi-agent-research-system

[46] Effective context engineering for AI agents — Anthropic Engineering (Applied AI team: Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, Jeremy Hadfield), published 29 Sep 2025. www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

AI Agents

Site Reliability Engineering

LLMOps

Platform Engineering

Software Architecture

PRODUCTION AI ENGINEERING · THE SERIES SO FAR

FOUNDATION

F1 · MCP Tool Sprawl

Your AI agent has 500 MCP tools. Now what?

FOUNDATION

F2 · Layered Architecture

Your agent works in a demo. Why does it break in production?

FOUNDATION

F3 · Headless AI

Your AI shouldn't live inside the UI

TRUST

T2 · Authorization & Policy

What is this agent actually allowed to do?

TRUST

T3 · Human-in-the-Loop

Authorized. Should it still act?

TRUST

T5 · Observability & Governance

Your AI agent did something in production. Can you explain exactly what happened?

TRUST

T4 · AI Control Plane

Your agents shouldn't govern themselves

STATE

S1 · Memory, Context & State

Your agent remembers everything. That's a problem.

TRUST

T1 · Agent Identity

Who is acting, and on whose authority?

TRUST

T6 · Agent, Tool & MCP Security

Assume the context is hostile.

RELIABILITY

R1 + R2 · Evals, Observability & Reliability

"The agent failed" is not an operational signal.

CAPSTONE

P1 · The Agent Is Not the Architecture

The final reference architecture

COORDINATION

C1 · Do You Actually Need Multiple Agents?

When does one agent become several, and when is that just more complexity?

CURRENT

O1 + O2 · Operating AI Agents at Scale

Cost, latency, scale & lifecycle.

The full learning map (Start Here)

O1 + O2 · Production AI Engineering

Production AI Engineering · O1 + O2. Every measured number here is substituted at build time from ops_poc/runs/2026-10-08-recorded/facts.json. The platform controls are real code; the clock, providers, tools, clients and agents are a deterministic simulation in simulated milliseconds and cost units, and the whole run replays byte for byte. Figures carry their provenance.

Medium edition Evidence Check POC README Run summary Claim evidence (JSON)
QA report